
Flow Equivariant Recurrent Neural Networks

T. Anderson Keller

The Kempner Institute for the Study of Natural and Artificial Intelligence
Harvard University, Cambridge, MA 15213
t.anderson.keller@gmail.com

Abstract

Data arrives at our senses as a continuous stream, smoothly transforming from one instant to the next. These smooth transformations can be viewed as continuous symmetries of the environment that we inhabit, defining equivalence relations between stimuli over time. In machine learning, neural network architectures that respect symmetries of their data are called equivariant and have provable benefits in terms of generalization ability and sample efficiency. To date, however, equivariance has been considered only for static transformations and feed-forward networks, limiting its applicability to sequence models, such as recurrent neural networks (RNNs), and corresponding time-parameterized sequence transformations. In this work, we extend equivariant network theory to this regime of ‘flows’ – one-parameter Lie subgroups capturing natural transformations over time, such as visual motion. We begin by showing that standard RNNs are generally not flow-equivariant: their hidden states fail to transform in a geometrically structured manner for moving stimuli. We then show how flow equivariance can be introduced, and demonstrate that these models significantly outperform their non-equivariant counterparts in terms of training speed, length generalization, and velocity generalization, on both next step prediction and sequence classification. We present this work as a first step towards building sequence models that respect the time-parameterized symmetries which govern the world around us.

1 Introduction

Every moment, the world around us shifts continuously. As you walk down the street, rooftops glide past, trees sway in the breeze, and cars drift in and out of view. These smooth, structured transformations – or flows – are not arbitrary, but instead are underpinned by a precise set of rules: an algebra which ensures self-consistency across space and time. Mathematically, these transformations which leave the essence of the world around you unchanged are called symmetries, and, in a sense, these symmetries over time can be seen to weave together the instantaneous into the continuous.

In machine learning, symmetry has emerged as a powerful handle from which we can grapple with the challenges of generalization and data efficiency. Models which encode symmetries are called ‘equivariant’, and their steady evolution has delivered significant performance improvements for data with known structure. Simultaneously, with the success of large language models, sequence modeling has become a prominent learning paradigm throughout machine learning sub-fields. We therefore ask: can these everyday smooth, continuous motion symmetries be encoded in our modern sequence models? Surprisingly, to date, the study of equivariance has largely been limited to temporally static transformations, narrowly missing the opportunity to capture the continuous time-parameterized symmetry transformations that dominate our natural experience. In this work, we consider a carefully structured subset of natural motion transformations called flows: formally, one-parameter Lie subgroups. We introduce a formal notion of what it means for a sequence model to be ‘flow equivariant’ and proceed to demonstrate that existing sequence models are indeed *not* flow equivariant, *even* when all sub-components of these models are individually equivariant with respect to the full ‘static’ Lie group symmetry.

Preprint. Under review.

Motivated by this gap in the literature, and the potential advantages of integrating one of the most ubiquitous natural symmetries into sequence models, in this work, we study how we might build Recurrent Neural Networks (RNNs) which are explicitly equivariant with respect to time-parameterized symmetry transformations. At the highest level, we demonstrate that we are able to do so in a manner very similar to Group equivariant Convolutional Neural Networks (G-CNNs): by lifting the hidden state of our RNN to a dimension of flows, indexed by the generating vector fields ν . In slices, our model resembles a bank of RNNs (one for each flow), but weight sharing across ν turns that bank into a single group-equivariant dynamical system which guarantees generalization to unseen flows.

In the following, we will review the principle of equivariance in neural network architectures (§2). We will formally introduce the notion of ‘Flow Equivariance’ (§3), and demonstrate that existing recurrent neural network architectures are indeed not ‘flow equivariant’ as we would desire. We will then introduce Flow Equivariant Recurrent Neural Networks (FERNNs) in §4; and demonstrate empirically that FERNNs achieve zero-shot generalization to new flows at test time, improved length generalization, and improved performance on datasets which possess strong flow symmetries in §5. In our discussion, (§6), we briefly outline related work on equivariance with respect to motion, and highlight the differences with our proposed approach, but leave a thorough review of related work to §D. We conclude with the limitations of our proposed framework and promising future directions.

2 Equivariant Neural Networks

Informally, a network is said to be equivariant with respect to a set of structured transformations if every transformation of the input has a corresponding structured transformation of the output. The fact that this set of transformations is ‘structured’ is important. Typically, this structure is the abstract algebraic structure known as a ‘group’ G , meaning that any two transformations $g_i, g_j \in G$ will combine through a binary operation $(\cdot)$ to form another known transformation $g_i \cdot g_j = g_k \in G$ (closure), and the ‘chunking’ of transformations is arbitrary, i.e. $(g_i \cdot g_j) \cdot g_k = g_i \cdot (g_j \cdot g_k)$ (associativity). As we will see later, we will exploit both these properties when designing equivariant models. At an intuitive level, an equivariant neural network Φ is a network whose activations ‘live on the group’. This means that when Φ processes a transformed input, by associativity, this is equivalent to transforming the activations, and by closure, this transformation just yields another set of activations on the group – a structured transformation of the output.

Formally, a neural network layer Φ is equivariant with respect to a symmetry group G if the action of the group commutes with the application of the layer. Following the group equivariant CNN literature [Cohen and Welling, 2016a], we will define this layer as a map between function spaces $\Phi : \mathcal{F}_K(X) \rightarrow \mathcal{F}_{K'}(Y)$, where $\mathcal{F}_K(X) := \{f : X \rightarrow \mathbb{R}^K\}$ denotes the set of signals defined on the abstract domain X with K channels. We will use the notation $\Phi[f]$ to denote the output of the model (in $\mathcal{F}_{K'}(Y)$ space) for an input signal f , and $\Phi[f](y) \in \mathbb{R}^{K'}$ to similarly denote the value of that output function at location y . For $f \in \mathcal{F}_K(X)$, we can then define the action of the group element $g \in G$ to be the left action with representation $\pi_X : G \rightarrow \text{Aut}(X)$ on the domain (where $\text{Aut}(X)$ is the automorphism group of all bijections $X \rightarrow X$), and a linear representation $\rho_X : G \rightarrow GL(K)$ on the channels: $(g \cdot f)(x) := \rho_X(g)f(\pi_X(g^{-1})x)$. Throughout the rest of this paper, we will only consider scalar feature types, meaning that the channel representation is trivial, $\rho_X(g) = \mathbf{I}_K$, and we will write the action of the input representation as $g \cdot x := \pi_X(g)x$ for brevity. Succinctly, this yields:

$$(g \cdot f)(x) := f(g^{-1} \cdot x), \quad g \in G, x \in X. \quad (1)$$

Concretely, if f is a 2D image, $X = \mathbb{Z}^2$ the set of pixel coordinates, and $G = (\mathbb{Z}^2, +)$ is the 2D translation group, then the translation action is: $(g \cdot f)(x) = f(x - g)$, a shift of pixel coordinates. In the output domain, the action of π_Y, ρ_Y depends on how we construct the equivariant map, but similarly has the form: $(g \cdot \Phi)(y) := \rho_Y(g)\Phi[f](\pi_Y(g^{-1})y)$. In this work, we will again only consider scalar features, meaning $\rho_Y(g) = \mathbf{I}_{K'}$. Furthermore, as we will show next, for group equivariant convolutional layers, $\pi_Y(g^{-1})$ acts the same as it does on the input, i.e. $(g \cdot \Phi[f])(y) = \Phi[f](g^{-1} \cdot y)$. Finally, the equivariance condition is then written simply as:

$$g \cdot \Phi[f] = \Phi[g \cdot f] \quad \forall g \in G, f \in \mathcal{F}_K(X) \quad (2)$$

We can see this is the formalization of our intuitive definition: the action of the group on the input f can instead be pulled through to act on the lifted group indices of the network’s output $\Phi[f]$.

Group Convolution. A simple example of such an equivariant map Φ is the group convolution [Cohen and Welling, 2016a]: a generalization of the standard convolutional neural network (CNN) layer from the translation group to other arbitrary discrete groups. To accomplish this, we define convolutional kernels to be functions on the group G , i.e. $\mathcal{W}^i : G \rightarrow \mathbb{R}^K$, and perform a ‘lifting convolution’ at the first layer of the network to ‘lift’ from the input space to the group space. Mathematically, we write the lifting convolution ($\hat{\star}_G$) and subsequent group convolutions ($\star_G$) as:

$$[f \hat{\star}_G \mathcal{W}^i](g) = \sum_{x \in \mathbb{Z}^2} \sum_{k=1}^K f_k(x) \mathcal{W}_k^i(g^{-1} \cdot x) \quad \& \quad [f \star_G \mathcal{W}^i](g) = \sum_{h \in G} \sum_{k=1}^K f_k(h) \mathcal{W}_k^i(g^{-1} \cdot h) \quad (3)$$

We see that the output of the layer is now defined over the group G , and we can verify that these layers are indeed equivariant with respect to Equation 2 (see §A.1). The important property being that $[(g \cdot f) \star_G \mathcal{W}^i] = g \cdot [f \star_G \mathcal{W}^i]$, that is, the convolution commutes with the group action.

Group Equivariant Recurrent Neural Networks. In this work we define a recurrent neural network h as a map between space-time function spaces $h : \mathcal{F}_K(X, \mathbb{Z}) \rightarrow \mathcal{F}_{K'}(Y, \mathbb{Z})$, where $\mathcal{F}_K(X, \mathbb{Z}) := \{f : X \times \mathbb{Z} \rightarrow \mathbb{R}^K\}$ denotes the set of time-varying K -dimensional signals defined over space X and (discretized) time $t \in \mathbb{Z}$. In the remainder of this paper we will use discrete time domains for our space-time signals, but we note that in order to accommodate continuous time sequence models, time may be made continuous ($t \in \mathbb{R}$) by changing domain of h , with only minor changes to the analysis. We can then define a group-equivariant RNN (G-RNN) as a simple RNN with group convolutions in place of both the usual linear recurrence and input maps. Explicitly:

$$h_{t+1}^i[f_{\leq t}] = \sigma(h_t[f_{< t}] \star_G \mathcal{W}^i + f_t \hat{\star}_G \mathcal{U}^i), \quad (4)$$

where $\mathcal{W}^i : G \rightarrow \mathbb{R}^K$, and $\mathcal{U}^i : G \rightarrow \mathbb{R}^K$ are the group convolutional kernels for the hidden-to-hidden mapping and input-to-hidden mapping respectively (for output channel i), and h_0 is initialized to be invariant to the group action, i.e. $h_0(g) = h_0(g') \forall g, g' \in G$. The notation $h_t[f_{< t}]$ implies that h_t is a causal function of the input signal f prior to time t . The non-linearity σ is assumed group-equivariant as well (pointwise). In §A.2, we prove by induction that this network is indeed equivariant with respect to action of the group G on the input signal f , meaning $h_t^i[g \cdot f_{\leq t}] = g \cdot h_t^i[f_{\leq t}] \forall g \in G, t \in \mathbb{Z}, f \in \mathcal{F}_K(X, \mathbb{Z})$. Intuitively, this is because both the input-to-hidden mapping and the recurrent mapping are equivariant, meaning any constant transformation of the input can be pulled through both mappings (and the nonlinearity) yielding a corresponding transformation of the output (hidden state). Concretely, this implies that a recurrent neural network with convolutional recurrent connections, and convolutional encoders is indeed still equivariant with respect to a static translation of the entire input sequence. However, as we will show next, *this does not imply that the RNN will be equivariant with respect to time-parameterized translation, i.e. motion.*

3 Flow Equivariance

Like the pathline traced out by a droplet of dye carried along by a moving fluid, the flow $\psi_t(\nu)$ generated by a vector field ν gives the position of any point after time $t \in \mathbb{R}$. More formally, a flow is a one-parameter subgroup of a Lie group G , generated by an element ν of the corresponding Lie algebra $\mathfrak{g}$ of G [Hall, 2015]. We denote a generator by $\nu \in \mathfrak{g}$, and its time- t flow $\psi_t(\nu) : \mathbb{R} \times \mathfrak{g} \rightarrow G$ by $\psi_t(\nu) = \exp(t\nu) \in G$, where $\exp$ is the exponential map. By construction, the flow admits an identity at $t = 0$, $\psi_0(\nu) = e$, and a composition property: $\psi_s(\nu) \cdot \psi_t(\nu) = \psi_{s+t}(\nu)$, such that flowing for time s and then t is the same as flowing for time $s + t$. Intuitively, the flow $\psi_t(\nu)$ is the group element that transports a point from the identity to where it will be after time t when moving with instantaneous velocity ν . Colloquially, we therefore call ψ a *time-parameterized symmetry transformation*. For a given $f \in \mathcal{F}_K(X, \mathbb{Z})$ and generator $\nu \in \mathfrak{g}$, we define a flow acting on f as

$$(\psi(\nu) \cdot f)_t(x) := f_t(\psi_t(\nu)^{-1} \cdot x). \quad (5)$$

and identically for a flow acting on a function in the output space $\mathcal{F}_{K'}(Y, \mathbb{Z})$: $(\psi(\nu) \cdot \Phi)_t(y) := \Phi_t(\psi_t(\nu)^{-1} \cdot y)$. Throughout this work we will use the notation $(\psi(\nu) \cdot f)$ and $(\psi(\nu) \cdot \Phi)$, leaving time implicit, to denote the flow acting on the entire time domain of the function simultaneously. We can then define ‘flow equivariance’ as the property of a sequence model such that its sequential output commutes with such time-parameterized transformations of the input:

Definition 3.1 (Flow Equivariance). For a set of generators $V \subseteq \mathfrak{g}$, a sequence model $\Phi : \mathcal{F}_K(X, \mathbb{Z}) \rightarrow \mathcal{F}_{K'}(Y, \mathbb{Z})$ is *equivariant with respect to the flows generated by V* iff

$$\psi(\nu) \cdot \Phi[f] = \Phi[\psi(\nu) \cdot f] \quad \forall \nu \in V, f \in \mathcal{F}_K(X, \mathbb{Z}). \quad (6)$$

Flow Equivariance of Sequence Models. Our proposed Definition 3.1 has a noteworthy special case which captures much of the literature to date on equivariance in sequence models. Specifically: *Remark* (Frame-wise feed-forward equivariant models are trivially flow equivariant.). Let $\Phi[f] = [\phi(f_0), \phi(f_1) \dots \phi(f_T)]$ be a frame-wise applied equivariant map, with $\phi : \mathcal{F}_K(X) \rightarrow \mathcal{F}_{K'}(Y)$ such that: $\phi(\psi_t(\nu) \cdot f_t) = \psi_t(\nu) \cdot \phi(f_t) \forall t \in \mathbb{Z}, \nu \in V$. Then Φ is flow equivariant by Definition 3.1.

See §A.3 for the short proof. Intuitively, one sees this very clearly for CNNs applied to video sequences frame-wise: since the CNN commutes with the transformation of each frame, and the transformation on the sequence distributes amongst the frames (by linearity of the action), the act of transforming the input frame-by-frame is equivalent to transforming the output frame-by-frame. Since this is achieved through a complete lack of sequence modeling in Φ , we call it ‘trivial flow equivariance’. Equivalently, one can see that the G-RNN defined in Equation 4 with $\mathcal{W} = \mathbf{0}$ reduces to such a feed-forward equivariant map (since recurrence is removed), and is thus also trivially flow equivariant. To achieve more sophisticated flow equivariance in the context of sequence processing, we must use sequence models with non-zero recurrence. Notably, while in §2 we show that group-equivariant RNNs do commute with static transformations, we show below that such models generally do not commute with time-parameterized transformations, meaning they are not flow equivariant.

Theorem 3.1 (G-RNNs are not flow equivariant). A group-equivariant RNN as defined in Equation 4, with non-zero $\mathcal{W}$, is not flow equivariant according to Definition 3.1, except in the degenerate flow-invariant case.

Remark. (Degenerate flow invariance) If all kernels $\mathcal{W}$ & $\mathcal{U}$ are constant on G , then the outputs of Equation 4 are spatially uniform. Since the flow action permutes only the G -index, these constants are fixed points, making the G-RNN flow invariant.

Proof Sketch. (Theorem 3.1) As visualized in Figure 1, let $\sigma = \text{Id}$, and $[\ast \star_G \mathcal{U}]$, $[\ast \star_G \mathcal{W}]$ be the identity, such that hidden state evolves simply as a sum of the past state and the new input: $h_{t+1}[f_{\leq t}] = h_t[f_{\leq t}] + f_t$. If the input is a static bump centered at 0, $h_{t+1}[f_{\leq t}]$ will be an ever growing bump, while $h_{t+1}[\psi(\nu) \cdot f_{\leq t}]$ will be a ‘train’ of bumps, since, at each time step, the hidden state is ‘lagging behind’ the input which has shifted by $\psi_1(\nu)$. Thus a corresponding shift of the growing bump will never equal the train of bumps, i.e. $(\psi(\nu) \cdot h[f_{\leq t}])_{t+1} \neq h_{t+1}[\psi(\nu) \cdot f_{\leq t}]$. $\square$

One can verify that this counter example applies to most other existing sequence models, including state space models, gated recurrent neural networks, neural ODES, and even transformers. In the following section, we show how to build a recurrent neural network which is equivariant with respect to these transformations, and argue that this same construction can likely be carried over to other sequence models as well.

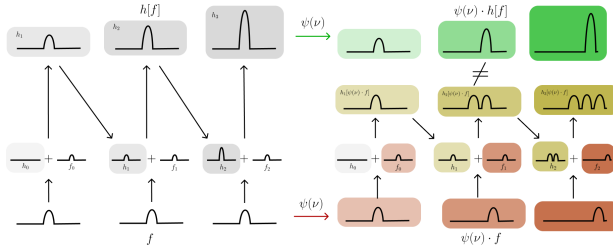


Figure 1: **G-RNNs are not generally flow equivariant.** We show this by simple counterexample with: $h_{t+1} = h_t + f_t$.

4 Flow Equivariant Recurrent Neural Networks

In this section we introduce our Flow Equivariant Recurrent Neural Network (FERNN) and prove the equivariance condition (Definition 3.1) holds. Inspired by the above analysis of the failure of G-RNNs to be flow equivariant, we note that a simple condition by which equivariance can be achieved is if computation is performed ‘in the reference frame’ of the signal. For the case of static symmetry groups, we can interpret the group convolution as performing the same operation (dot product of filter and signal) in all possible statically transformed reference frames (group elements). In the case of time-parameterized symmetry groups, we can then assume that we desire to be performing the same recurrent update in all possible dynamically transformed (moving) reference frames. To achieve this, we see that, analogous to the g -lift in the G-CNN, we must now lift our hidden state and input to the flow elements $\nu \in V$. To do so, we define the action of the instantaneous group elements $\psi_t(\nu) \in G$

on the lifted $V \times G$ space to be the same left action as before, only on the G dimension. Explicitly, for $h_t(\nu, g) : V \times G \rightarrow \mathbb{R}^{K'}$, we define $\psi_t(\nu) \cdot h_t(\nu, g) := h_t(\nu, \psi_t(\nu)^{-1} \cdot g)$.

Input Flow-Lifting Convolution. To lift the input to the product group $V \times G$, we will re-use the original lifting group convolution from Equation 3 exactly, and simply perform a ‘trivial lift’ to the additional V dimension by copying the output of the convolution for each value of ν . Explicitly:

$$[f \hat{\star}_{V \times G} \mathcal{U}^i](\nu, g) = \sum_{x \in X} \sum_{k=1}^K f_k(x) \mathcal{U}_k^i(g^{-1} \cdot x) \quad (7)$$

In §A.5 we prove that this is equivariant with respect to the group action, yielding the following:

$$\psi_t(\nu) \cdot [f_t \hat{\star}_{V \times G} \mathcal{U}^i](\nu, g) = [(\psi_t(\nu) \cdot f_t) \hat{\star}_{V \times G} \mathcal{U}^i](\nu, g) := [f_t \hat{\star}_{V \times G} \mathcal{U}^i](\nu, \psi_t(\nu)^{-1} \cdot g) \quad (8)$$

We note that it is also possible to perform a non-trivial lift, where $\mathcal{U}$ is defined on the lifted space $V \times G$, as in G-CNNs. This changes the later recurrence relation (Eqn. 11) necessary to achieve flow equivariance, but is largely a design choice. In the experiments and proofs we stick with the trivial lift, but describe the model with non-trivial lift in §C.1.

Flow Convolution. The group convolution can then be extended to our lifted hidden state as:

$$[h \star_{V \times G} \mathcal{W}^i](\nu, g) = \sum_{\gamma \in V} \sum_{m \in G} \sum_{k=1}^{K'} h_k(\gamma, m) \mathcal{W}_k^i(\gamma - \nu, g^{-1} \cdot m). \quad (9)$$

We see that for the convolution over V , we have assumed that $V \subseteq \mathfrak{g}$ is an additive (abelian) discrete group, such that the combination $(-\nu) \cdot \gamma = \gamma - \nu$. This is equivalent to asserting that the action of the flow generator on the lifted hidden state is given by the shift: $(\hat{\nu} \cdot h_t)(\nu, g) = h_t(\nu - \hat{\nu}, g)$. If V is not abelian, the general Baker-Campbell-Hausdorff product $((-\nu) \cdot \gamma)$ should be used instead [Hall, 2015]. Similarly, if V is not discrete, one should perform an integral with respect to the Haar measure. In all experiments we use abelian V , and a discrete grid for V , so this is exact. In §A.6 we prove this is equivariant with respect to the action of individual flow elements. Explicitly:

$$[(\psi_t(\nu) \cdot h) \star_{V \times G} \mathcal{W}^i](\nu, g) = \psi_t(\nu) \cdot [h \star_{V \times G} \mathcal{W}^i](\nu, g) \quad (10)$$

Intuitively, this is simply because the individual group elements along the flow only act on the G coordinate of the signal, and thereby commute with the standard group convolution.

Flow Equivariant Recurrence Relation. We define the recurrence relation for the FERNN as:

$$h_{t+1}(\nu, g) = \sigma(\psi_1(\nu) \cdot [h_t \star_{V \times G} \mathcal{W}](\nu, g) + [f_t \hat{\star}_{V \times G} \mathcal{U}](\nu, g)). \quad (11)$$

We see that there are two primary differences between this model and the G-RNN: the extra dimension ν , and the action of the instantaneous one-step flow element $\psi_1(\nu)$ at each position ν of the lifted hidden state. Intuitively, this can be understood as a bank of G-RNNs, each flowing autonomously according to their corresponding vector fields ν simultaneously. Similar to G-CNNs then, when an input arrives undergoing a given flow transformation with generator $\hat{\nu}$, this construction allows us to ‘pull out’ this transformation into a shift of the ν dimension. Formally:

Theorem 4.1. (FERNNs are flow equivariant) Let $h[f] \in \mathcal{F}_{K'}(Y, \mathbb{Z})$ be a FERNN as defined in Equations 7, 9, and 11, with hidden-state initialization invariant to the group action and constant in the flow dimension, i.e. $h_0(\nu, g) = h_0(\nu', g) \forall \nu', \nu \in V$ and $\psi_1(\nu) \cdot h_0(\nu, g) = h_0(\nu, g) \forall \nu \in V, g \in G$. Then, $h[f]$ is flow equivariant according to Definition 3.1 with the following re-defined representation of the action of the flow in the output space:

$$(\psi(\hat{\nu}) \cdot h[f])_t(\nu, g) = h_t[f](\nu - \hat{\nu}, \psi_{t-1}(\hat{\nu})^{-1} \cdot g) \quad (12)$$

In words, the equivariance condition $h_t[\psi(\hat{\nu}) \cdot f](\nu, g) = h_t[f](\nu - \hat{\nu}, \psi_{t-1}(\hat{\nu})^{-1} \cdot g)$ means that the hidden state of the FERNN processing a flowing input sequence has a corresponding flow along the G dimension, and a shift along the V dimension, both determined by the flow generator $\hat{\nu}$. In §A.7 we prove Theorem 4.1 by induction, and give a brief sketch below.

Proof Sketch. (Theorem 4.1) We wish to prove $h_t[\psi(\hat{\nu}) \cdot f](\nu, g) = h_t[f](\nu - \hat{\nu}, \psi_{t-1}(\hat{\nu})^{-1} \cdot g) \quad \forall t$. Letting $H = V \times G$, we write the recurrence relation for the transformed input as:

$$h_{t+1}[\psi(\hat{\nu}) \cdot f](\nu, g) = \sigma(\psi_1(\nu) \cdot [h_t[\psi(\hat{\nu}) \cdot f] \star_H \mathcal{W}](\nu, g) + [(\psi_t(\hat{\nu}) \cdot f_t) \star_H \mathcal{U}](\nu, g)). \quad (13)$$

Assume we initialize the hidden state to a fixed point of the flow-element action $\psi_1(\nu) \cdot h_0(\nu, g) = h_0(\nu, g)$, constant in ν . We can see that the base case is satisfied trivially since $\psi_1(\nu) \cdot h_0(\nu, g) = \psi_{-1}(\nu) \cdot h_0(\nu - \hat{\nu}, g)$ by constancy. We then assert the inductive hypothesis, allowing us to substitute $h_t[\psi(\hat{\nu}) \cdot f](\nu, g) \Rightarrow h_t[f](\nu - \hat{\nu}, \psi_{t-1}(\hat{\nu})^{-1} \cdot g) = \psi_{t-1}(\hat{\nu}) \cdot h_t[f](\nu - \hat{\nu}, g)$ into equation 13. Similarly, due to the trivial lift and the equivariance of lifting convolution, we can substitute $[(\psi_t(\hat{\nu}) \cdot f_t) \star_{V \times G} \mathcal{U}](\nu, g) \Rightarrow \psi_t(\hat{\nu}) \cdot [f_t \star_{V \times G} \mathcal{U}](\nu - \hat{\nu}, g)$. Ultimately, this gives us:

$$(13) = \sigma(\psi_1(\nu) \cdot \psi_{t-1}(\hat{\nu}) \cdot [h_t[f_{<t}] \star_H \mathcal{W}](\nu - \hat{\nu}, g) + \psi_t(\hat{\nu}) \cdot [f_t \star_H \mathcal{U}](\nu - \hat{\nu}, g)). \quad (14)$$

By the properties of flows, we know that $\psi_1(\nu) \cdot \psi_{t-1}(\hat{\nu}) = \psi_1(\nu) \cdot \psi_{-1}(\hat{\nu}) \cdot \psi_t(\hat{\nu}) = \psi_t(\hat{\nu}) \cdot \psi_1(\nu - \hat{\nu})$, and thus we can pull out a factor of $\psi_t(\hat{\nu})$ from both terms, leaving us with the original recurrence defined over a new V index $\nu - \hat{\nu}$, exactly the shift in V . $\square$

Intuitively, the idea is that we would like to build a sequence model which fixes the lack of equivariance demonstrated by the counter example in Figure 1. To accomplish this, we note that the hidden state of the model in Figure 1 appears to be lagging behind behind the input at each time step. To fix this, we therefore propose to augment our hidden state with a bank of flows, each generated by a separate vector field ν such that when the input arrives at a specific velocity, it will exactly match one of these flow parameters, effectively shifting the ‘zero’ point of the moving coordinates of our RNN to this new flow. Since these flows are relative quantities (obeying the group law), we see that all the other ν channels in the bank will similarly shift according to their relative position in V space (e.g. relative velocity). These effects combined lead to the output representation we observe analytically.

We finally note that since the hidden state lives on the product group $V \times G$, and the input is similarly lifted to the product group, one can simply stack these layers, replacing the lifting convolution with another flow convolution and maintain flow equivariance. Since a composition of equivariant maps is itself an equivariant map [Kondor and Trivedi, 2018], a deep FERNN is also flow equivariant (provided all non-linearities, normalization layers, and other additional architectural components are additionally flow equivariant – we refer readers to [Bronstein et al., 2021] for an extensive review).

Practical Implementation. We note that exact equivariance requires that the group V be closed under the action of the flow on the output. In theory this can be satisfied by many constructions, but in practice we cannot lift our hidden state to an infinite dimensional group on a machine with finite memory. Therefore, similar to prior work [Worrall and Welling, 2019], we implement a truncated version of the group V , and incur equivariance error at the boundaries. Furthermore, in the majority of experiments in this work, we define the flow convolution to not mix the V channels at all, explicitly: $\mathcal{W}_k^i(\nu, g) = \delta_{\nu=e} \mathcal{W}_k^i(g)$. We find that this is beneficial to not propagate errors between the ν channels induced by the truncation, similar to prior work on scale equivariance [Sosnovik et al., 2020b].

5 Experiments

In the following subsections we introduce a suite of datasets with known flow symmetries and study how adding flow equivariance impacts performance compared with non-equivariant baselines. Specifically, we investigate two small scale sequence datasets: (i) next-step prediction on the MNIST dataset with imposed translation and rotation flows [LeCun et al., 1998], and (ii) sequence classification on the KTH human action recognition dataset [Schuldt et al., 2004], where we augment the dataset with additional translation flows to simulate camera motion. In §B we include the full details of the dataset creation, model architectures, training, and evaluation procedures; and in §E we include extended results. Code is available at: <https://github.com/Neurips-FERNN/FERNN>.

Flowing MNIST Datasets. We construct a number of variants of the Flowing MNIST dataset to test generalization, each defined by the set of generators we use for training and evaluation (V_{train} , V_{test}). For rotation flows, we discretize the rotation algebra $\mathfrak{so}(2)$ at $\Delta\theta = 10^\circ$ intervals, yielding: $V_N^R = \{k\Delta\theta J | k = -N, \dots, N\}$, where $J = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix} \in \mathfrak{so}(2)$. For 2D translation flows, we always

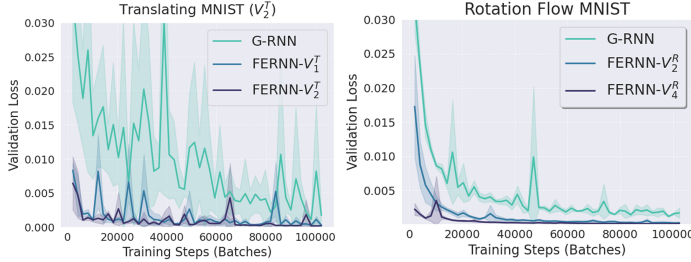


Figure 2: **Increased flow equivariance increases training speed on data with flow symmetry.** Validation loss vs. train steps.

Model	Test MSE ↓
$V_{train} = V_{test} = V_2^T$	
G-RNN	$8.1e-4 \pm 1e-4$
FERNN- V_1^T	$1.9e-4 \pm 5e-5$
FERNN- V_2^T	$1.1e-4 \pm 2e-5$
$V_{train} = V_{test} = V_4^R$	
G-RNN	$1.1e-3 \pm 4e-5$
FERNN- V_3^R	$2.5e-4 \pm 3e-5$
FERNN- V_4^R	$9.7e-5 \pm 9e-6$

Table 1: **Flowing MNIST Test MSE.** (mean $\pm$ std, 3 seeds)

use the integer lattice up to velocities N_{step}^{pixels} : $V_N^T = \{\nu \in \mathbb{Z}^2 \mid \|\nu\|_\infty \leq N\}$. See Figures 3 & 4 for examples. A train/test example from the dataset is a sequence f_t constructed by drawing a random static sample f from the original MNIST dataset, drawing a random vector field $\hat{\nu}$ uniformly from the corresponding V_* , and applying the action of the flow to generate the timeseries: $f_t = \psi_t(\hat{\nu}) \cdot f$.

Next-Step Prediction Task. Models are trained to solve an autoregressive forward prediction task: given the first $T = 10$ time-steps of the timeseries f , predict the next 10 time-steps. We use teacher-forcing with 50% probability during training to ensure training convergence. Model parameters are optimized to minimize the mean-squared error (MSE) between predictions and the ground truth for 50 epochs. In the length generalization experiments of Figure 3, we evaluate the model’s MSE on forward prediction up to 30 steps, significantly longer than the 10 seen in training.

Next-Step Prediction Models. We use G-RNNs and FERNNs exactly as defined in Equations 4 & 11 respectively, where G is defined as $SO(2)$ with translations ($SE(2)$) for the rotation flow datasets, and $G = (\mathbb{Z}^2, +)$ is the standard 2D translation group for the translation flow datasets. We use the *escnn* library to implement the $SE(2)$ convolutions [Cesa et al., 2022]. We set $\sigma = \text{ReLU}$, use a single recurrent layer, a hidden state size K' of 128 channels, and (3×3) convolutional kernels in the spatial dimensions. We note that the spatial dimensions of the input are preserved throughout the entire forward pass, i.e. there is no spatial pooling, stride, or linear layers. We augment these models with a small 4-layer CNN decoder $g_\theta(h_{t+1})$ with 128 channels, and ReLU activations, which maps from each hidden state to the corresponding output. Explicitly: $g_\theta(h_{t+1}) = \hat{f}_{t+1}$, where the training loss is the MSE between $\hat{f}_{t+1}$ and f_{t+1} averaged over time $t \in [11, 20]$. For the FERNN models, we also crucially ‘max-pool’ over the V dimension of the hidden state before decoding, e.g. $g_\theta(\max_\nu h_{t+1}(\nu, g)) = \hat{f}_{t+1}$. This means that all G-RNN and FERNN models have exactly the same number of parameters, since the FERNN can be seen to share its parameters over the different flow elements. For each FERNN, we denote its corresponding set of generators with FERNN- V_N^* .

Next-Step Performance. In Figure 2 we show the validation loss curves for these models on Flowing MNIST, and in Table 1 we show the final MSE on the test sets (mean $\pm$ std. over 3 random initializations). We see that on both datasets, FERNNs dramatically outperform G-RNNs, reaching nearly an order of magnitude lower error, and doing so in significantly fewer iterations. We also see that even partial equivariance (e.g. FERNN- V_1^T on V_2^T) is highly beneficial to model performance.

Length Generalization In Figure 3 we plot generated forward predictions on Translation-Flow MNIST with $V_{train} = V_{test} = V_2^T$ for both the G-RNN and FERNN- V_2^T models compared with the ground truth sequence. The length of sequences seen in training is indicated by the shaded gray area, and we ask models to generate significantly longer than that. We see that the G-RNN predictions begin to degrade after the training length, while the FERNN remains highly consistent. We quantify this error for each forward prediction step T in the adjacent plot, demonstrating that indeed FERNNs dramatically outperform G-RNNs in length generalization. We note that the performance improvement of FERNNs over G-RNNs is more significant for translation flows compared with rotation flows in our experiments. Particularly, we do not see quite as strong length generalization (see §E); however, we speculate that this is due to the interpolation procedure necessary to implement small rotations on a finite grid, not due to any difference in the underlying theory. This interpolation is known to break exact equivariance, and we suggest prior methods for addressing this issue in G-CNNs may be equally applicable to FERNNs [Diaconu and Worrall, 2019].

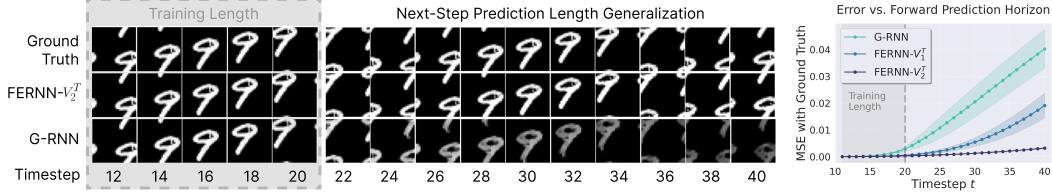


Figure 3: **FERNNs exhibit next-step prediction length-generalization capabilities far surpassing G-RNNs on simple flows.** We plot samples for the forward prediction trajectories of a G-RNN and FERNN- V_2^T trained on Translating MNIST V_2^T to predict 10-steps into the future (down-sampled by a factor of 2 in time for visualization). We see the G-RNN performs well on this training regime but diverges rapidly with lengths longer than training. The FERNN generalizes nearly perfectly. On the right, we plot this forward prediction error vs. the forward prediction timestep for three models.

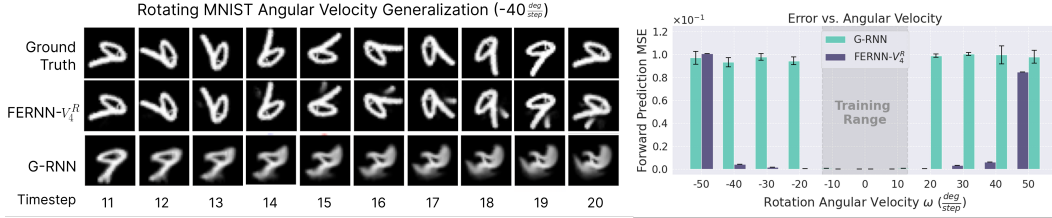


Figure 4: **FERNNs exhibit next-step prediction generalization to previously unseen rotation flow velocities where G-RNNs fail.** Samples of forward predictions for FERNN- V_4^R trained on Rotating MNIST with V_1^R and tested on V_5^R . We see the FERNN- V_4^R achieves near perfect forward-prediction performance in this range. On the right we plot the full distribution of error across V_{test} .

Velocity Generalization Similar to standard group equivariant neural networks, weight sharing induces automatic generalization to previously unseen group elements. In the case of flow equivariance, we then expect zero-shot generalization to flow generators that are part of the lifted hidden state’s V but not seen during training. In Figures 4 & 5, we show example sequences which demonstrate this to be the case. Specifically, the models are trained on V_2^R and V_1^T , and evaluated on the full set of generators from V_4^R and V_2^T respectively. In the adjacent plots, we show the test MSE for sequences flowing according to each generator. We see that the FERNNs (middle) generalize nearly perfectly, while the G-RNNs (bottom) only learn to forward predict in-domain flows, and fails to generalize outside.

Moving KTH Action Recognition. Finally, we test our FERNN model’s ability to classify natural video sequences with induced motion using the simple task of Action Recognition from the KTH dataset. The dataset is composed of videos of 25 people performing 6 different actions. We use the traditional train-val-test split, by person, requiring strong generalization to new individuals. We down-sample each clip by a factor of 2 in space and time for computational convenience, leading to grayscale sequences f of size 32×32 , with 16 time-steps each. In our experiments, we apply flows from V_1^T and V_2^T to the dataset as an attempt to emulate a slow camera motion.

Action Recognition Models. In Figure 6 and Table 2, we compare the accuracy of FERNN models with non-flow-equivariant counterparts (G-RNN, G-RNN+, & 3D-CNN), each with comparable parameter counts. The G-RNN and FERNN- V_*^T models are identical to those used on the Translating MNIST dataset with the addition of a 3-Layer CNN encoder (in place of the input convolution $\mathcal{U}$), and a class-readout module composed of an average-pooling over the spatial dimensions followed by a linear layer. The G-RNN+ model is an ablation of the FERNN- V_2^T which has the same lifted hidden state and max-pooling over the lifted dimensions, but instead of applying the transformation $\psi_1(\nu)$ to each hidden index ν , the G-RNN+ learns its own

Model	Test Acc. $\uparrow$	# θ
3D-CNN	0.626 ± 0.02	209K
G-RNN+	0.639 ± 0.02	242K
G-RNN	0.665 ± 0.03	242K
FERNN- V_1^T	0.698 ± 0.03	242K
FERNN- V_2^T	0.716 ± 0.04	242K

Table 2: **FERNNs outperform non-equivariant models on sequence classification in the presence of strong motion.** Test accuracy (mean $\pm$ std) for models trained and tested on the Moving KTH dataset (V_2^T).

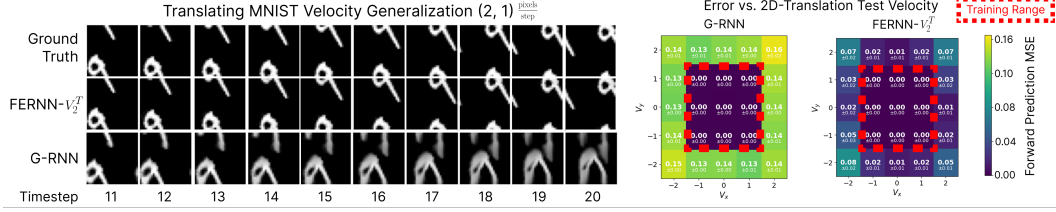


Figure 5: **FERNNs exhibit next-step prediction generalization to previously unseen translation flow velocities where G-RNNs fail.** Samples of forward prediction trajectories for FERNN- V_2^T trained on Translating MNIST with V_1^T and tested on V_2^T . On the right we plot the full distribution of error across the 2D velocity coordinates in V_{test} .

transformation through a small convolutional kernel (see §B). The inclusion of this model ensures that the benefits that we see from the FERNN are not simply due to having an increased hidden state size and an extra max-pooling non-linearity – instead they must be coming from the flow on the hidden state itself. The 3D-CNN is a 5-layer network composed of 3D convolutions that performs striding and pooling to eventually yield a 64-dimensional feature vector which is fed to a linear classification head.

Action Recognition Results We see in Figure 6 that the FERNN models again outperform non-flow-equivariant models on generalization to classifying actions undergoing new flows at test time (plots highlighted in orange). We note that the drop in performance is likely due to the V truncation discussed in §4 breaking exact equivariance and causing overfitting. In Table 2, we see that when trained and tested on strongly moving data, $V_{train} = V_{test} = V_2^T$, the FERNN significantly outperforms the non-flow-equivariant counterparts with no train-test disparity.

6 Discussion

In this work we have introduced flow equivariance, a general notion of equivariance with respect to time-parameterized group transformations for sequence models, and provided an initial framework for constructing sequence models which satisfy this definition.

Related Work. While we believe our definition of flow equivariance and our FERNN construction are novel, we emphasize that we are far from the first to consider equivariance with respect to motion groups. Specifically, a number of works have developed equivariance with respect to space-time transformations such as Lorentz transformations [Bogatskiy et al., 2020, Gong et al., 2022]. These works essentially propose to process the input as a 4D spacetime block such that the transformation is therein self-contained. The core novelty of our work compared to this literature is the fact that our approach applies to causal sequence processing, and is defined for arbitrary sequence models rather than spatio-temporal convolutions only. A number of works consider equivariance in recurrent frameworks, [Azari and Erdoğmuş, 2022, Nguyen et al., 2023] but only consider static coordinate transformations, not those which change over time, as our flows. Closest to our work, ‘Equivariant Observer Theory’ from the control theory literature [Mahony et al., 2022] has studied how to build hand-crafted models and filters which are indeed equivariant with respect to time-parameterized symmetries; however this work lacks a notion of training familiar to the geometric deep learning community. Prior work in the computational neuroscience community has studied the idea of equivariance with respect to temporal scaling [Zuo et al., 2023] and 1D translation [Zhang et al., 2022] in biologically plausible recurrent neural network circuits. This work is highly relevant and perhaps in future work it may be possible to derive such models as

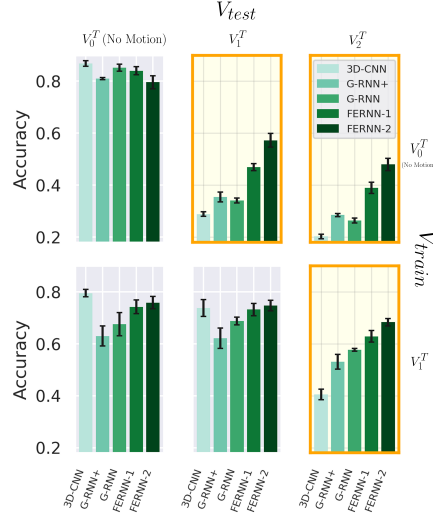


Figure 6: **FERNNs outperform non-equivariant models on action recognition undergoing out-of-distribution flows (orange).** Test accuracy of models trained on KTH Action Recognition with no motion V_0^T (top row) and trained with data augmented by flow generators V_1^T (bottom row). The models are tested on data augmented by flow generators V_0^T, V_1^T, V_2^T (columns).

special cases of our proposed FERNN, or as novel extensions. Notably, several biologically inspired sequence models, including the Topographic VAE [Keller and Welling, 2022] and Neural Wave Machine [Keller and Welling, 2023], explicitly ‘roll’ latent representations so that a single-velocity flow is baked into the network and subsequently learned from data. Our formulation suggests that these heuristics may be special cases of flow-equivariant RNNs and generalizes the principle to any set of continuous flows V . We refer readers to §D for an extended discussion on related work.

Limitations & Future Work. The greatest limitation of the proposed FERNN construction is that it relies on the shift representation of the flow action on the hidden state. This implies that the number of activations must linearly increase with the size of V , and additionally induces the truncation errors described in §4. Future work on an analog to Steerable CNNs [Cohen and Welling, 2016b] for flow equivariance (if possible) would be highly valuable in making these models more computationally efficient. A second limitation of the current FERNN model is the simplicity of the network architecture (being built as a simple RNN). However, since the flow lift and convolution are linear, time-invariant operations, a lifted state-space layer is still a linear convolution along time, and all of the computational shortcuts exploited by modern SSMs (FFT kernels, associative scans) should still apply. Finally, convolutional RNNs (such as FERNNs) are becoming increasingly of interest in the computational neuroscience and ‘NeuroAI’ domains [Spoerer et al., 2017], with notable examples being trained as ‘foundation models’ for mouse visual cortex [Wang et al., 2025], and recently demonstrating state of the art performance in modeling rodent somatosensory cortex [Chung et al., 2025]. Our work provides a new theoretical lens through which to study and build such models through the lens of geometry, offering potential novel insights into biological neural systems.

Acknowledgments

We would like to thank Max Welling for continued guidance and advice, which ultimately led the author to the development of this work. We would also like to thank Maurice Weiler, John Vastola, and Gyu Heo for discussions which improved this work. We would like to thank members of the CRISP Lab at Harvard, and the Kempner Institute for the opportunity to present and improve early versions of the work. Finally, we would like to thank the Kempner Institute for funding, and for the opportunity to independently pursue the development of this work through the author’s research fellowship.

References

- Bahar Azari and Deniz Erdoğmuş. Equivariant deep dynamical model for motion prediction, 2022. URL <https://arxiv.org/abs/2111.01892>.
- D. Barch, T.N.Vijaykumar Kumar, and D. Glaser. Modeling the illusory motion of enigma with an excitable neuronal array. *Journal of Vision - J VISION*, 3:72–72, 12 2010. doi: 10.1167/3.12.72.
- Davis Barch and Donald Glaser. Synaptic depression and facilitation can induce motion aftereffects in an excitable membrane model of visual motion processing. *Neurocomputing*, 44-46:51–59, 06 2002. doi: 10.1016/S0925-2312(02)00346-6.
- Sourya Basu, Prasanna Sattigeri, Karthikeyan Natesan Ramamurthy, Vijil Chenthamarakshan, Kush R. Varshney, Lav R. Varshney, and Payel Das. Equi-tuning: group equivariant fine-tuning of pretrained models. In *Proceedings of the Thirty-Seventh AAAI Conference on Artificial Intelligence and Thirty-Fifth Conference on Innovative Applications of Artificial Intelligence and Thirteenth Symposium on Educational Advances in Artificial Intelligence*, AAAI’23/IAAI’23/EAAI’23. AAAI Press, 2023. ISBN 978-1-57735-880-0. doi: 10.1609/aaai.v37i6.25832. URL <https://doi.org/10.1609/aaai.v37i6.25832>.
- Alexander Bogatskiy, Brandon Anderson, Jan T. Offermann, Marwah Roussi, David W. Miller, and Risi Kondor. Lorentz group equivariant neural network for particle physics, 2020. URL <https://arxiv.org/abs/2006.04780>.
- Michael M. Bronstein, Joan Bruna, Yann LeCun, Arthur Szlam, and Pierre Vandergheynst. Geometric deep learning: Grids, groups, graphs, geodesics, and gauges. *arXiv preprint arXiv:2104.13478*, 2021.
- Gabriele Cesa, Leon Lang, and Maurice Weiler. A program to build E(N)-equivariant steerable CNNs. In *International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=WE4qe9x1nQw>.

- S Chemla, A Reynaud, M di Volo, Y Zerlaut, L Perrinet, A Destexhe, and F Chavane. Cortical suppressive waves shape the representation of long-range apparent motion. *bioRxiv*, 2018. doi: 10.1101/372763. URL <https://www.biorxiv.org/content/early/2018/10/09/372763>.
- Christopher Choy, JunYoung Gwak, and Silvio Savarese. 4d spatio-temporal convnets: Minkowski convolutional neural networks, 2019. URL <https://arxiv.org/abs/1904.08755>.
- Trinity Chung, Yuchen Shen, Nathan C. L. Kong, and Aran Nayebi. Task-optimized convolutional recurrent networks align with tactile processing in the rodent brain, 2025. URL <https://arxiv.org/abs/2505.18361>.
- Taco Cohen and Max Welling. Group equivariant convolutional networks. In Maria Florina Balcan and Kilian Q. Weinberger, editors, *Proceedings of The 33rd International Conference on Machine Learning*, volume 48 of *Proceedings of Machine Learning Research*, pages 2990–2999, New York, New York, USA, 20–22 Jun 2016a. PMLR. URL <https://proceedings.mlr.press/v48/cohenc16.html>.
- Taco S. Cohen and Max Welling. Steerable cnns, 2016b. URL <https://arxiv.org/abs/1612.08498>.
- Nichita Diaconu and Daniel E Worrall. Learning to convolve: A generalized weight-tying approach, 2019. URL <https://arxiv.org/abs/1905.04663>.
- Carlos Esteves, Christine Allen-Blanchette, Xiaowei Zhou, and Kostas Daniilidis. Polar transformer networks, 2018. URL <https://arxiv.org/abs/1709.01889>.
- Fynn Fromme, Christine Allen-Blanchette, Hans Harder, and Sebastian Peitz. Surrogate modeling of 3d rayleigh-benard convection with equivariant autoencoders, 2025. URL <https://arxiv.org/abs/2505.13569>.
- Mathias Gehrig and Davide Scaramuzza. Recurrent vision transformers for object detection with event cameras, 2023. URL <https://arxiv.org/abs/2212.05598>.
- Donald A Glaser and Davis Barch. Motion detection and characterization by an excitable membrane: The “bow wave” model. *Neurocomputing*, 26-27:137–146, 1999. ISSN 0925-2312. doi: [https://doi.org/10.1016/S0925-2312\(99\)00067-3](https://doi.org/10.1016/S0925-2312(99)00067-3). URL <https://www.sciencedirect.com/science/article/pii/S0925231299000673>.
- Shiqi Gong, Qi Meng, Jue Zhang, Huilin Qu, Congqiao Li, Sitian Qian, Weitao Du, Zhi-Ming Ma, and Tie-Yan Liu. An efficient lorentz equivariant graph neural network for jet tagging. *Journal of High Energy Physics*, 2022(7), July 2022. ISSN 1029-8479. doi: 10.1007/jhep07(2022)030. URL [http://dx.doi.org/10.1007/JHEP07\(2022\)030](http://dx.doi.org/10.1007/JHEP07(2022)030).
- Jonathan Gordon, David Lopez-Paz, Marco Baroni, and Diane Bouchacourt. Permutation equivariant models for compositional generalization in language. In *International Conference on Learning Representations*, 2020. URL <https://openreview.net/forum?id=SylVNerFvr>.
- Deepak K. Gupta, Devanshu Arya, and Efstratios Gavves. Rotation equivariant siamese networks for tracking, 2020. URL <https://arxiv.org/abs/2012.13078>.
- Brian C. Hall. *Lie Groups, Lie Algebras, and Representations: An Elementary Introduction*. Springer International Publishing, 2015. ISBN 9783319134673. doi: 10.1007/978-3-319-13467-3. URL <http://dx.doi.org/10.1007/978-3-319-13467-3>.
- Geoffrey Hinton, Sara Sabour, and Nicholas Frosst. Matrix capsules with em routing. 2018. URL <https://openreview.net/pdf?id=HJWLfGWRb>.
- Max Jaderberg, Karen Simonyan, Andrew Zisserman, and Koray Kavukcuoglu. Spatial transformer networks, 2016. URL <https://arxiv.org/abs/1506.02025>.
- T Anderson Keller. Nu-wave state space models: Traveling waves as a biologically plausible context. *Science Communications Worldwide*, 2025. doi: 10.57736/b30b-8eed. URL <https://www.world-wide.org/cosyne-25/nu-wave-state-space-models-traveling-3803805f>.
- T. Anderson Keller and Max Welling. Topographic vaes learn equivariant capsules, 2022. URL <https://arxiv.org/abs/2109.01394>.
- T. Anderson Keller and Max Welling. Neural wave machines: Learning spatiotemporally structured representations with locally coupled oscillatory recurrent neural networks. In Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett, editors, *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 16168–16189. PMLR, 23–29 Jul 2023. URL <https://proceedings.mlr.press/v202/keller23a.html>.

- T. Anderson Keller, Lyle Muller, Terrence Sejnowski, and Max Welling. Traveling waves encode the recent past and enhance sequence learning, 2024a. URL <https://arxiv.org/abs/2309.08045>.
- T. Anderson Keller, Lyle Muller, Terrence J. Sejnowski, and Max Welling. A spacetime perspective on dynamical computation in neural information processing systems, 2024b. URL <https://arxiv.org/abs/2409.13669>.
- Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization, 2017. URL <https://arxiv.org/abs/1412.6980>.
- Risi Kondor and Shubhendu Trivedi. On the generalization of equivariance and convolution in neural networks to the action of compact groups. In *International Conference on Machine Learning (ICML)*, 2018.
- Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. In *Proceedings of the IEEE*, volume 86, pages 2278–2324, November 1998. doi: 10.1109/5.726791.
- Alejandro López-Cifuentes, Marcos Escudero-Viñolo, and Jesús Bescós. A prospective study on sequence-driven temporal sampling and ego-motion compensation for action recognition in the epic-kitchens dataset, 2020. URL <https://arxiv.org/abs/2008.11588>.
- Robert Mahony, Pieter van Goor, and Tarek Hamel. Observer design for nonlinear systems with equivariance. *Annual Review of Control, Robotics, and Autonomous Systems*, 5(Volume 5, 2022):221–252, 2022. ISSN 2573-5144. doi: <https://doi.org/10.1146/annurev-control-061520-010324>. URL <https://www.annualreviews.org/content/journals/10.1146/annurev-control-061520-010324>.
- J. Moehlis and E. Knobloch. Equivariant dynamical systems. *Scholarpedia*, 2(10):2510, 2007. doi: 10.4249/scholarpedia.2510. revision #91240.
- Hai Huu Nguyen, Andrea Baisero, David Klee, Dian Wang, Robert Platt, and Christopher Amato. Equivariant reinforcement learning under partial observability. In *7th Annual Conference on Robot Learning*, 2023. URL <https://openreview.net/forum?id=AnDDMQgM7->.
- Sara Sabour, Nicholas Frosst, and Geoffrey E Hinton. Dynamic routing between capsules, 2017. URL <https://arxiv.org/abs/1710.09829>.
- Christian Schuldt, Ivan Laptev, and Barbara Caputo. Recognizing human actions: A local svm approach. In *Pattern Recognition, 2004. ICPR 2004. Proceedings of the 17th International Conference on*, volume 3, pages 32–36. IEEE Computer Society, 2004. ISBN 0-7695-2128-2. doi: 10.1109/ICPR.2004.747.
- Sage Shaw and Zachary P Kilpatrick. Representing stimulus motion with waves in adaptive neural fields, 2023. URL <https://arxiv.org/abs/2312.06100>.
- Ivan Sosnovik, Artem Moskalev, and Arnold Smeulders. Scale equivariance improves siamese tracking, 2020a. URL <https://arxiv.org/abs/2007.09115>.
- Ivan Sosnovik, Michał Szmaja, and Arnold Smeulders. Scale-equivariant steerable networks, 2020b. URL <https://arxiv.org/abs/1910.11093>.
- Courtney J. Spoerer, Patrick McClure, and Nikolaus Kriegeskorte. Recurrent convolutional neural networks: A better model of biological object recognition. *Frontiers in Psychology*, Volume 8 - 2017, 2017. ISSN 1664-1078. doi: 10.3389/fpsyg.2017.01551. URL <https://www.frontiersin.org/journals/psychology/articles/10.3389/fpsyg.2017.01551>.
- Søren Kaae Sønderby, Casper Kaae Sønderby, Lars Maaløe, and Ole Winther. Recurrent spatial transformer networks, 2015. URL <https://arxiv.org/abs/1509.05329>.
- Eric Y. Wang, Paul G. Fahey, Zhuokun Ding, Stelios Papadopoulos, Kayla Ponder, Marissa A. Weis, Andersen Chang, Taliah Muhammad, Saumil Patel, Zhiwei Ding, Dat Tran, Jiakun Fu, Casey M. Schneider-Mizell, Nuno Maçarico da Costa, R. Clay Reid, Forrest Collman, Katrin Franke, Alexander S. Ecker, Jacob Reimer, Xaq Pitkow, Fabian H. Sinz, Andreas S. Tolias, and MICrONS Consortium. Foundation model of neural activity predicts response to new stimulus types. *Nature*, 640(8058):470–477, 2025. doi: 10.1038/s41586-025-08829-y. URL <https://doi.org/10.1038/s41586-025-08829-y>.
- Rui Wang, Robin Walters, and Rose Yu. Incorporating symmetry into deep dynamics models for improved generalization, 2021. URL <https://arxiv.org/abs/2002.03061>.
- Ronald J. Williams and David Zipser. A learning algorithm for continually running fully recurrent neural networks. *Neural Computation*, 1(2):270–280, 1989.

- Daniel E. Worrall and Max Welling. Deep scale-spaces: Equivariance over scale, 2019. URL <https://arxiv.org/abs/1905.11697>.
- Haixu Wu, Zhiyu Yao, Jianmin Wang, and Mingsheng Long. Motionrnn: A flexible model for video prediction with spacetime-varying motions, 2021. URL <https://arxiv.org/abs/2103.02243>.
- Theodoros P. Zanos, Patrick J. Mineault, Konstantinos T. Nasiatou, Daniel Guitton, and Christopher C. Pack. A sensorimotor role for traveling waves in primate visual cortex. *Neuron*, 85(3):615–627, 2015. ISSN 0896-6273. doi: <https://doi.org/10.1016/j.neuron.2014.12.043>. URL <https://www.sciencedirect.com/science/article/pii/S0896627314011520>.
- Mingming Zhang, Rajat S. Shivacharan, Chia-Chu Chiang, Luis E. Gonzalez-Reyes, and Dominique M. Durand. Propagating neural source revealed by doppler shift of population spiking frequency. *Journal of Neuroscience*, 36(12):3495–3505, 2016. ISSN 0270-6474. doi: 10.1523/JNEUROSCI.3525-15.2016. URL <https://www.jneurosci.org/content/36/12/3495>.
- Wenhao Zhang, Ying Nian Wu, and Si Wu. Translation-equivariant representation in recurrent networks with a continuous manifold of attractors. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems*, volume 35, pages 15770–15783. Curran Associates, Inc., 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/file/65384a01325fecbd364c835db872443c-Paper-Conference.pdf.
- Maksim Zhdanov, David Ruhe, Maurice Weiler, Ana Lucic, Johannes Brandstetter, and Patrick Forré. Clifford-steerable convolutional neural networks. In Ruslan Salakhutdinov, Zico Kolter, Katherine Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp, editors, *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 61203–61228. PMLR, 21–27 Jul 2024. URL <https://proceedings.mlr.press/v235/zhdanov24a.html>.
- Alex Zihao Zhu, Ziyun Wang, and Kostas Daniilidis. Motion equivariant networks for event cameras with the temporal normalization transform, 2019. URL <https://arxiv.org/abs/1902.06820>.
- Yuliang Zhu, Jing Cheng, Zhuo-Xu Cui, Jianfeng Ren, Chengbo Wang, and Dong Liang. Sre-cnn: A spatiotemporal rotation-equivariant cnn for cardiac cine mr imaging, 2024. URL <https://arxiv.org/abs/2409.08537>.
- Junfeng Zuo, Xiao Liu, Ying Nian Wu, Si Wu, and Wenhao Zhang. A recurrent neural circuit mechanism of temporal-scaling equivariant representation. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. URL <https://openreview.net/forum?id=K9dmkfZcMu>.

Appendices

Table of Contents

A	Extended Proofs	22
A.1	Equivariance of Group-Lifting Convolution & Group-Convolution	22
A.2	Equivariance of Group Equivariant RNN	23
A.3	Frame-wise Flow Equivariance	23
A.4	Group Equivariant RNNs are not Flow Equivariant	24
A.5	Equivariance of Lifting Flow Convolution	26
A.6	Equivariance of Flow Convolution	26
A.7	Equivariance of FERNN	26
B	Experiment Details	28
B.1	Flowing MNIST: Dataset Creation	28
B.2	Flowing MNIST: Models	29
B.3	Flowing MNIST: Next-Step Prediction Training & Evaluation	30
B.4	Moving KTH: Datasets	31
B.5	Moving KTH: Models	31
B.6	Moving KTH: Action Recognition Training & Evaluation	32
B.7	Moving KTH Samples	32
B.8	Compute	33
C	Variations of FERNNs	34
C.1	FERNN with Non-Trivial Lift	34
D	Related Work	35
D.1	Flow Equivariance without a Hidden State	35
D.2	‘Statically Equivariant’ Sequence Models	35
D.3	Neuroscience and Biologically Inspired Neural Networks	35
D.4	Reference Frames in Neural Networks	36
D.5	Broadly Related Work	36
D.6	Equivariant Dynamical Systems	37
E	Extended Results: Flowing MNIST	38
E.1	In-Distribution Next-Step Prediction (Table 1 & Figure 2)	38
E.2	Length Generalization Visualizations	38
E.3	Velocity Generalization Visualizations	40

A Extended Proofs

A.1 Equivariance of Group-Lifting Convolution & Group-Convolution

We can verify that group convolutional layers given by Equation 3 are equivariant according to Equation 2. While these are well-known facts, we re-derive them here in our own notation for completeness and to lay the groundwork for our later proofs. First, for the lifting convolution, we wish to prove:

$$[(\hat{g} \cdot f) \hat{\star}_G \mathcal{W}^i](g) = \hat{g} \cdot [f \hat{\star}_G \mathcal{W}^i](g) \quad \forall g, \hat{g} \in G, f \in \mathcal{F}_K(X). \quad (15)$$

Proof. (Group-Lifting Conv. is Group Equivariant)

$$[(\hat{g} \cdot f) \hat{\star}_G \mathcal{W}^i](g) = \sum_{x \in \mathbb{Z}^2} \sum_{k=1}^K f_k(\hat{g}^{-1} \cdot x) \mathcal{W}_k^i(g^{-1} \cdot x) \quad (\text{by def. action, Eqn. 1}) \quad (16)$$

$$= \sum_{\hat{x} \in \mathbb{Z}^2} \sum_{k=1}^K f_k(\hat{x}) \mathcal{W}_k^i(g^{-1} \cdot (\hat{g} \cdot \hat{x})) \quad (\text{where } \hat{x} = \hat{g}^{-1} \cdot x) \quad (17)$$

$$= \sum_{\hat{x} \in \mathbb{Z}^2} \sum_{k=1}^K f_k(\hat{x}) \mathcal{W}_k^i((g^{-1} \cdot \hat{g}) \cdot \hat{x}) \quad (\text{by associativity}) \quad (18)$$

$$= \sum_{\hat{x} \in \mathbb{Z}^2} \sum_{k=1}^K f_k(\hat{x}) \mathcal{W}_k^i((\hat{g}^{-1} \cdot g)^{-1} \cdot \hat{x}) \quad (\text{by defn. inverse}) \quad (19)$$

$$= [f \hat{\star}_G \mathcal{W}^i](\hat{g}^{-1} \cdot g) = \hat{g} \cdot [f \hat{\star}_G \mathcal{W}^i](g) \quad (\text{by Eqn. 3 \& defn. action}) \quad (20)$$

□

In line 17, we use the fact that because the group G acts on X by a bijection (see our earlier definition of the input representation $\pi_X : G \rightarrow \text{Aut}(X)$ in §2), the substitution $\hat{x} = g^{-1} \cdot x$ is just a relabeling of the index set, and hence the sum over $x \in X$ is the same as the sum over $\hat{x} \in X$.

Next for the group convolution, the proof is virtually identical. We wish to prove:

$$[(\hat{g} \cdot f) \star_G \mathcal{W}^i](g) = \hat{g} \cdot [f \star_G \mathcal{W}^i](g) \quad \forall g, \hat{g} \in G, f \in \mathcal{F}_K(X). \quad (21)$$

Proof. (Group-Conv. is Group Equivariant)

$$[(\hat{g} \cdot f) \star_G \mathcal{W}^i](g) = \sum_{h \in G} \sum_{k=1}^K f_k(\hat{g}^{-1} \cdot h) \mathcal{W}_k^i(g^{-1} \cdot h) \quad (\text{by def. action, Eqn. 1}) \quad (22)$$

$$= \sum_{\hat{h} \in G} \sum_{k=1}^K f_k(\hat{h}) \mathcal{W}_k^i(g^{-1} \cdot (\hat{g} \cdot \hat{h})) \quad (\text{where } \hat{h} = \hat{g}^{-1} \cdot h) \quad (23)$$

$$= \sum_{\hat{h} \in G} \sum_{k=1}^K f_k(\hat{h}) \mathcal{W}_k^i((g^{-1} \cdot \hat{g}) \cdot \hat{h}) \quad (\text{by associativity}) \quad (24)$$

$$= \sum_{\hat{h} \in G} \sum_{k=1}^K f_k(\hat{h}) \mathcal{W}_k^i((\hat{g}^{-1} \cdot g)^{-1} \cdot \hat{h}) \quad (\text{by defn. inverse}) \quad (25)$$

$$= [f \star_G \mathcal{W}^i](\hat{g}^{-1} \cdot g) = \hat{g} \cdot [f \star_G \mathcal{W}^i](g) \quad (\text{by Eqn. 3 \& defn. action}) \quad (26)$$

□

Again, we use the fact that the group G is closed under the group action, so the substitution $\hat{h} = \hat{g}^{-1} \cdot h$ is just a relabeling of the index set, and hence the sum over $h \in G$ is the same as the sum over $\hat{h} \in G$.

A.2 Equivariance of Group Equivariant RNN

We wish to prove that a recurrent neural network built with group-equivariant convolutional layers (Equation 4) is equivariant according to Equation 2. If we write the hidden state as a function of the input signal $h_{t+1}[f_{\leq t}](g) \in \mathcal{F}_{K'}(Y)$, where $Y = G$, then we can write the the equivariance condition as:

$$h_{t+1}[\hat{g} \cdot f_{\leq t}](g) = \hat{g} \cdot h_{t+1}[f_{\leq t}](g) \quad \forall t \in \mathbb{Z}_+, f \in \mathcal{F}_K(X), \hat{g}, g \in G, \quad (27)$$

where $\hat{g} \cdot f_{\leq t} := \{\hat{g} \cdot f_i \mid i \in \mathbb{Z}_+ \leq t\}$, denotes the input signal generated by applying the same group element to each timestep. We prove this by induction for all $t \in \mathbb{Z}_+$:

Proof. (G-RNN is Group Equivariant)

We assume (i) $\hat{\star}_G$ and $\star_G$ are the group-lifting convolution and group convolution defined in Equation 3, (ii) σ is a G-equivariant non-linearity (e.g. pointwise), and (iii) g acts linearly on h -space. Since h is defined by the lifting and recurrent convolutions, we see that $Y = G$. We further assume (iv) h_0 is initialized to be invariant to the group action, i.e. $\hat{g} \cdot h_0(g) = h_0(\hat{g}^{-1} \cdot g) = h_0(g)$, and (v) the input signal is zero before time zero, i.e. $f_{<0} = \mathbf{0}$.

Base Case: We can see that the base case is trivially true from the initial condition, since the initial condition is not a function of the input sequence, so:

$$h_0[\hat{g} \cdot f_{<0}](g) = h_0[f_{<0}](g) \quad (\text{since } h_0 \text{ is indep. of input } f_{<0}) \quad (28)$$

$$= h_0[f_{<0}](\hat{g}^{-1} \cdot g) \quad (\text{by initialization}) \quad (29)$$

$$= \hat{g} \cdot h_0[f_{<0}](g) \quad (\text{by defn. action}) \quad (30)$$

Inductive Step: Assuming $h_t[\hat{g} \cdot f_{<t}](g) = \hat{g} \cdot h_t[f_{<t}](g)$, we wish to prove this holds also for $t + 1$:

$$h_{t+1}[\hat{g} \cdot f_{\leq t}](g) = \sigma([h_t[\hat{g} \cdot f_{<t}] \star_G \mathcal{W}](g) + [\hat{g} \cdot f_t] \hat{\star}_G \mathcal{U})(g) \quad (\text{by defn. G-RNN}) \quad (31)$$

$$= \sigma([\hat{g} \cdot h_t[f_{<t}]] \star_G \mathcal{W})(g) + [\hat{g} \cdot f_t] \hat{\star}_G \mathcal{U}(g) \quad (\text{by inductive hyp.}) \quad (32)$$

$$= \sigma(\hat{g} \cdot [h_t[f_{<t}] \star_G \mathcal{W}](g) + \hat{g} \cdot [f_t \hat{\star}_G \mathcal{U}](g)) \quad (\text{by equivar. of G-conv.}) \quad (33)$$

$$= \hat{g} \cdot \sigma([h_t[f_{<t}] \star_G \mathcal{W}](g) + [f_t \hat{\star}_G \mathcal{U}](g)) \quad (\text{by equivar. non-lin.}) \quad (34)$$

$$= \hat{g} \cdot h_{t+1}[f_{\leq t}](g) \quad (\text{by defn. G-RNN, Eqn 4}) \quad (35)$$

□

We note that the step on the fourth line assumes that the group action is linear, in that it distributes over the addition operation between the previous hidden state and the input. Traditionally, group equivariant neural network architectures have been constructed to exhibit linear representations in the latent space, and therefore this is a natural assumption. However, if one allowed g to act by non-linear maps (i.e. dropped the linear-representation assumption), then neither distributivity nor σ -equivariance would hold in general.

A.3 Frame-wise Flow Equivariance

We wish to prove that any G-equivariant map $\phi : \mathcal{F}_K(X) \rightarrow \mathcal{F}_{K'}(Y)$, that is applied ‘frame-wise’ to a space-time function $f \in \mathcal{F}_K(X, \mathbb{Z})$ is also flow equivariant according to Definition 3.1. Specifically, let $\Phi[f] = [\phi(f_0), \phi(f_1) \dots \phi(f_T)]$ be a sequence model built by concatenating the output of a G-equivariant map ϕ applied to each timestep t of the input signal f_t . Furthermore, let ϕ be equivariant to the individual group elements generated by vector fields $\nu \in V$, i.e. $\phi(\psi_t(\nu) \cdot f_t) = \psi_t(\nu) \cdot \phi(f_t) \forall t \in \mathbb{Z}, \nu \in V, f_t \in \mathcal{F}_K(X)$. Then:

Proof. (Frame-wise Equivariant Maps are Flow Equivariant)

$$\Phi(\psi(\nu) \cdot f) = [\phi(\psi_0(\nu) \cdot f_0), \phi(\psi_1(\nu) \cdot f_1) \dots \phi(\psi_T(\nu) \cdot f_T)] \quad (\text{by defn. flow action}) \quad (36)$$

$$= [\psi_0(\nu) \cdot \phi(f_0), \psi_1(\nu) \cdot \phi(f_1) \dots \psi_T(\nu) \cdot \phi(f_T)] \quad (\text{by G-equivariance of } \phi) \quad (37)$$

$$= \psi(\nu) \cdot [\phi(f_0), \phi(f_1) \dots \phi(f_T)] \quad (\text{by defn. flow action, eqn. 5}) \quad (38)$$

$$= (\psi(\nu) \cdot \Phi(f)) \quad (\text{by defn. } \Phi) \quad (39)$$

□

A.4 Group Equivariant RNNs are not Flow Equivariant

In this subsection we prove Theorem 3.1, that the group-equivariant RNN as defined in Equation 4, with non-zero $\mathcal{W}$, is not flow equivariant according to Definition 3.1, except in the degenerate flow invariant case.

We will prove this in two parts: (i) First we will prove that a G-RNN with constant kernels is indeed flow invariant through induction (the degenerate case). Then (ii), we will proceed with a proof by contradiction to show that this is the only such flow equivariant G-RNN possible.

To begin, we recall that the degenerate flow invariant case is defined as a G-RNN where both $\mathcal{W}$ and $\mathcal{U}$ are constant over G . We will denote such kernels $\bar{\mathcal{U}}$ and $\bar{\mathcal{W}}$, such that:

$$\bar{\mathcal{W}}(g) = \bar{\mathcal{W}}(g'), \quad \forall g, g' \in G \quad \& \quad \bar{\mathcal{U}}(g \cdot x) = \bar{\mathcal{U}}(g' \cdot x) \quad \forall g, g' \in G, x \in X. \quad (40)$$

We wish to prove that such a network satisfies Definition 3.1, but more specifically that it is actually invariant to the flow action, meaning:

$$h_t[\psi(\nu) \cdot f_{<t}] = h_t[f_{<t}] \quad \forall f \in \mathcal{F}_K(X), \nu \in V, t \in \mathbb{Z}_+ \quad (41)$$

We can see that this is a special case of Definition 3.1 where the group action on the output space is trivial, i.e. given by the identity: $\psi_t(\nu) \cdot h_t[f_{<t}] = h_t[f_{<t}]$. We can prove this by induction as before. First we introduce the following lemma for convenience:

Lemma A.1. (Lifting convolution with $\bar{\mathcal{U}}$ is group invariant) The result of applying the lifting convolution ($\hat{\star}_G$) defined in 3 with a constant kernel $\bar{\mathcal{U}}$ to a signal $f \in \mathcal{F}_K(X)$ is invariant to flow element action on f , i.e.

$$[(\psi_t(\nu) \cdot f) \hat{\star}_G \bar{\mathcal{U}}^i](g) = [f \hat{\star}_G \bar{\mathcal{U}}^i](g) \quad \forall \nu \in V, f \in \mathcal{F}_K(X). \quad (42)$$

Proof. (Lemma A.1)

$$[(\psi_t(\nu) \cdot f) \hat{\star}_G \bar{\mathcal{U}}^i](g) = \sum_{x \in \mathbb{Z}^2} \sum_{k=1}^K f_k(\psi_t(\nu)^{-1} \cdot x) \bar{\mathcal{U}}_k^i(g^{-1} \cdot x) \quad (\text{by def. action, Eqn. 1}) \quad (43)$$

$$= \sum_{\hat{x} \in \mathbb{Z}^2} \sum_{k=1}^K f_k(\hat{x}) \bar{\mathcal{U}}_k^i(g^{-1} \cdot (\psi_t(\nu) \cdot \hat{x})) \quad (\text{where } \hat{x} = \psi_t(\nu)^{-1} \cdot x) \quad (44)$$

$$= \sum_{\hat{x} \in \mathbb{Z}^2} \sum_{k=1}^K f_k(\hat{x}) \bar{\mathcal{U}}_k^i((\psi_t(\nu)^{-1} \cdot g)^{-1} \cdot \hat{x}) \quad (\text{by associativity and inv.}) \quad (45)$$

$$= \sum_{\hat{x} \in \mathbb{Z}^2} \sum_{k=1}^K f_k(\hat{x}) \bar{\mathcal{U}}_k^i(\hat{g}^{-1} \cdot \hat{x}) \quad (\text{by closure, } \hat{g} = \psi_t(\nu)^{-1} \cdot g \in G) \quad (46)$$

$$= \sum_{\hat{x} \in \mathbb{Z}^2} \sum_{k=1}^K f_k(\hat{x}) \bar{\mathcal{U}}_k^i(g^{-1} \cdot \hat{x}) \quad (\text{by defn } \bar{\mathcal{U}}(g^{-1} \cdot \hat{x}) = \bar{\mathcal{U}}(\hat{g}^{-1} \cdot \hat{x})) \quad (47)$$

$$= [f \hat{\star}_G \bar{\mathcal{U}}^i](g) \quad (\text{by Eqn. 3}) \quad (48)$$

□

We have again used the fact that the group acts on X by a bijection as in §A.1. We can then proceed with the proof by induction to prove that the G-RNN with constant kernels is flow invariant:

Proof. (G-RNN with Constant Kernels is Flow Invariant)

Let $h_t[f_{<t}]$ be defined as in Equation 4 with $\mathcal{U} = \bar{\mathcal{U}}$ and $\mathcal{W} = \bar{\mathcal{W}}$. We again assume (i) $\hat{\star}_G$ and $\star_G$ are the group-lifting convolution and group convolution defined in Equation 3, (ii) σ is a G-equivariant non-linearity (e.g. pointwise), (iii) g acts linearly on h -space, (iv) h_0 is initialized to be invariant to the group action of the individual flow elements, i.e. $\psi_t(\nu) \cdot h_0(g) = h_0(\psi_t(\nu)^{-1} \cdot g) = h_0(g)$, and (v) the input signal is zero before time zero, i.e. $f_{<0} = 0$.

Base Case: We can see that the base case is again trivially true from the initial condition, since the initial condition is not a function of the input sequence, so:

$$h_0[\psi(\nu) \cdot f_{<0}](g) = h_0[f_{<0}](g) \quad (\text{since } h_0 \text{ is indep. of input } f_{<0}) \quad (49)$$

Inductive Step: Assuming the inductive hypothesis that $h_t[\psi(\nu) \cdot f_{<t}] = h_t[f_{<t}]$, we wish to prove this holds for $t + 1$:

$$h_{t+1}[\psi(\nu) \cdot f_{\leq t}](g) = \sigma([h_t[\psi(\nu) \cdot f_{<t}] \star_G \bar{\mathcal{W}}](g) + [\psi_t(\nu) \cdot f_t] \hat{\star}_G \bar{\mathcal{U}}](g)) \quad (\text{by defn. G-RNN}) \quad (50)$$

$$= \sigma([h_t[f_{<t}] \star_G \bar{\mathcal{W}}](g) + [\psi_t(\nu) \cdot f_t] \hat{\star}_G \bar{\mathcal{U}}](g)) \quad (\text{by inductive hyp.}) \quad (51)$$

$$= \sigma([h_t[f_{<t}] \star_G \bar{\mathcal{W}}](g) + [f_t \hat{\star}_G \bar{\mathcal{U}}](g)) \quad (\text{by Lemma A.1}) \quad (52)$$

$$= h_{t+1}[f_{\leq t}](g) \quad (\text{by eqn. 4}) \quad (53)$$

□

Finally then, we use a proof by contradiction to show that this degenerate flow invariant case is the only possible case for the G-RNN to be flow equivariant.

Proof. (Theorem 3.1, G-RNNs are not Generally Flow Equivariant)

Assert the converse, that the G-RNN defined in Equation 4 is flow equivariant, and that the kernels $\mathcal{W}$ and $\mathcal{U}$, are not constant, i.e. $\mathcal{W} \neq \bar{\mathcal{W}}$ & $\mathcal{U} \neq \bar{\mathcal{U}}$ (as defined in Equation 40). Then, it should be the case that:

$$h_{t+1}[\psi(\nu) \cdot f_{\leq t}](g) = (\psi(\nu) \cdot h[f_{\leq t}])_{t+1}(g) = h_{t+1}[f_{\leq t}](\psi_t(\nu)^{-1} \cdot g) \quad (54)$$

For $t = 0$ we have the following:

$$h_1[\psi(\nu) \cdot f_{\leq 0}](g) = \sigma([h_0[\psi(\nu) \cdot f_{<0}] \star_G \bar{\mathcal{W}}](g) + [\psi_0(\nu) \cdot f_0] \hat{\star}_G \bar{\mathcal{U}}](g)) \quad (55)$$

$$= \sigma([h_0[f_{<0}] \star_G \bar{\mathcal{W}}](g) + [f_0 \hat{\star}_G \bar{\mathcal{U}}](g)) \quad (\text{by defn. } h_0 \text{ \& } \psi_0) \quad (56)$$

$$= h_1[f_{\leq 0}](g) \quad (57)$$

For the next step, we get:

$$h_2[\psi(\nu) \cdot f_{\leq 2}](g) = \sigma([h_1[\psi(\nu) \cdot f_{\leq 0}] \star_G \bar{\mathcal{W}}](g) + [\psi_1(\nu) \cdot f_1] \hat{\star}_G \bar{\mathcal{U}}](g)) \quad (58)$$

$$= \sigma([h_1[f_{\leq 0}] \star_G \bar{\mathcal{W}}](g) + \psi_1(\nu) \cdot [f_1 \hat{\star}_G \bar{\mathcal{U}}](g)) \quad (\text{by above \& g-conv.}) \quad (59)$$

If we look at the action of the flow on the output space, according to flow equivariance, we should have:

$$(\psi(\nu) \cdot h[f_{\leq 2}])_2(g) = \sigma(\psi_1(\nu) \cdot [h_1[f_{\leq 0}] \star_G \bar{\mathcal{W}}](g) + [\psi_1(\nu) \cdot f_1] \hat{\star}_G \bar{\mathcal{U}}](g)) \quad (60)$$

$$= \sigma(\psi_1(\nu) \cdot [h_1[f_{\leq 0}] \star_G \bar{\mathcal{W}}](g) + \psi_1(\nu) \cdot [f_1 \hat{\star}_G \bar{\mathcal{U}}](g)) \quad (\text{by g-conv.}) \quad (61)$$

Setting $h_2[\psi(\nu) \cdot f_{\leq 2}](g) = (\psi(\nu) \cdot h[f_{\leq 2}])_2(g)$, as per our assumption, we see the input terms match exactly, but the h_{t-1} terms imply the following equivalence:

$$[h_1[f_{\leq 0}] \star_G \bar{\mathcal{W}}](g) = \psi_1(\nu) \cdot [h_1[f_{\leq 0}] \star_G \bar{\mathcal{W}}](g) \quad (62)$$

$$= [h_1[f_{\leq 0}] \star_G \bar{\mathcal{W}}](\psi_1(\nu)^{-1} \cdot g) \quad (\text{by action defn.}) \quad (63)$$

We see this is precisely the ‘lagging’ hidden state that we visualized in Figure 1. In order for this equality to hold for all $\nu \in V$, the output of the convolution must be constant along the flows generated by all ν . We can see that this is only satisfied by $\mathcal{W} = \bar{\mathcal{W}}$, a contradiction.

□

A.5 Equivariance of Lifting Flow Convolution

In this section we verify that the flow-lifting convolution given by Equation 7 is equivariant to action of the individual flow elements, with the following representation of the action:

$$\psi_t(\nu) \cdot [f_t \star_{V \times G} \mathcal{U}^i](\nu, g) = [(\psi_t(\nu) \cdot f_t) \star_{V \times G} \mathcal{U}^i](\nu, g) := [f_t \star_{V \times G} \mathcal{U}^i](\nu, \psi_t(\nu)^{-1} \cdot g) \quad (64)$$

Since the flow lifting convolution is a trivial lift, equivalent to the group-lifting convolution with an extra duplicated index, this proof is a trivial replication of the group-lifting convolution proof:

Proof. (Flow-Lifting Conv. is Flow Equivariant)

$$[(\psi_t(\nu) \cdot f_t) \star_{V \times G} \mathcal{U}^i](\nu, g) = \sum_{x \in X} \sum_{k=1}^K f_k(\psi_t(\nu)^{-1} \cdot x) \mathcal{U}_k^i(g^{-1} \cdot x) \quad (\text{by defn. action}) \quad (65)$$

$$= \sum_{\hat{x} \in X} \sum_{k=1}^K f_k(\hat{x}) \mathcal{U}_k^i(g^{-1} \cdot (\psi_t(\nu) \cdot \hat{x})) \quad (\text{where } \hat{x} = \psi_t(\nu)^{-1} \cdot x) \quad (66)$$

$$= \sum_{\hat{x} \in X} \sum_{k=1}^K f_k(\hat{x}) \mathcal{U}_k^i((\psi_t(\nu)^{-1} \cdot g)^{-1} \cdot \hat{x}) \quad (\text{by associativity \& inv.}) \quad (67)$$

$$= [f_t \star_{V \times G} \mathcal{U}^i](\nu, (\psi_t(\nu)^{-1} \cdot g)) \quad (68)$$

□

A.6 Equivariance of Flow Convolution

In this section, we prove that the flow convolution in Equation 9 is equivariant to the action of the individual flow elements yielding:

$$[(\psi_t(\nu) \cdot h) \star_{V \times G} \mathcal{W}^i](\nu, g) = \psi_t(\nu) \cdot [h \star_{V \times G} \mathcal{W}^i](\nu, g) \quad (69)$$

Proof. (Flow Conv. is Flow Equivariant)

$$[(\psi_t(\nu) \cdot h) \star_{V \times G} \mathcal{W}^i](\nu, g) = \sum_{\gamma \in V} \sum_{m \in G} \sum_{k=1}^{K'} h_k(\gamma, \psi_t(\nu)^{-1} \cdot m) \mathcal{W}_k^i(\gamma - \nu, g^{-1} \cdot m) \quad (70)$$

$$(\text{where } \hat{m} = \psi_t(\nu)^{-1} \cdot m) = \sum_{\gamma \in V} \sum_{\hat{m} \in G} \sum_{k=1}^{K'} h_k(\gamma, \hat{m}) \mathcal{W}_k^i(\gamma - \nu, g^{-1} \cdot (\psi_t(\nu) \cdot \hat{m})) \quad (71)$$

$$(\text{by associativity \& inverse}) = \sum_{\gamma \in V} \sum_{\hat{m} \in G} \sum_{k=1}^{K'} h_k(\gamma, \hat{m}) \mathcal{W}_k^i(\gamma - \nu, (\psi_t(\nu)^{-1} \cdot g)^{-1} \cdot \hat{m}) \quad (72)$$

$$(\text{by Eqn. 9}) = [h \star_{V \times G} \mathcal{W}^i](\nu, \psi_t(\nu)^{-1} \cdot g) \quad (73)$$

$$(\text{by action}) = \psi_t(\nu) \cdot [h \star_{V \times G} \mathcal{W}^i](\nu, g) \quad (74)$$

□

A.7 Equivariance of FERNN

In this section, we prove Theorem 4.1 by induction. We restate the theorem below:

Theorem. (FERNNs are flow equivariant) Let $h[f] \in \mathcal{F}_{K'}(Y, \mathbb{Z})$ be a FERNN as defined in Equations 7, 9, and 11, with hidden-state initialization invariant to the group action and constant in the flow dimension, i.e. $h_0(\nu, g) = h_0(\nu', g) \forall \nu', \nu \in V$ and $\psi_1(\nu) \cdot h_0(\nu, g) = h_0(\nu, g) \forall \nu \in V, g \in G$. Then, $h[f]$ is flow equivariant according to Definition 3.1 with the following representation of the action of the flow in the output space for $t \geq 1$:

$$(\psi(\hat{\nu}) \cdot h[f])_t(\nu, g) = h_t[f](\nu - \hat{\nu}, \psi_{t-1}(\hat{\nu})^{-1} \cdot g) \quad (75)$$

We note for the sake of completeness, that this then implies the follow equivariance relations:

$$h_t[\psi(\hat{\nu}) \cdot f](\nu, g) = h_t[f](\nu - \hat{\nu}, \psi_{t-1}(\hat{\nu})^{-1} \cdot g) = \psi_{t-1}(\hat{\nu}) \cdot h_t[f](\nu - \hat{\nu}, g) \quad (76)$$

Proof. (Theorem 4.1, FERNs are flow equivariant)

Identical to the G-RNN, we assume that σ is a G-equivariant non-linearity, g acts linearly on h -space, $h_0(\nu, g)$ is defined constant as above, and the input signal is zero before time zero, i.e. $f_{<0} = \mathbf{0}$.

Base Case: The base case is trivially true from the initial condition:

$$h_0[\psi(\hat{\nu}) \cdot f_{<0}](\nu, g) = h_0[f_{<0}](\nu, g) \quad (\text{by initial cond.}) \quad (77)$$

$$= h_0[f_{<0}](\nu - \hat{\nu}, \psi_{t-1}(\hat{\nu})^{-1} \cdot g) \quad (\text{by constant init.}) \quad (78)$$

Inductive Step: Assuming $h_t[\psi(\hat{\nu}) \cdot f](\nu, g) = h_t[f](\nu - \hat{\nu}, \psi_{t-1}(\hat{\nu})^{-1} \cdot g) \forall \nu \in V, g \in G$, for some $t \geq 0$, we wish to prove this also holds for $t + 1$:

Using the FERNN recurrence (Eqn. 11) on the transformed input, and letting $H = V \times G$, we get:

$$h_{t+1}[\psi(\hat{\nu}) \cdot f](\nu, g) = \sigma\left(\psi_1(\nu) \cdot [h_t[\psi(\hat{\nu}) \cdot f_{<t}] \star_H W](\nu, g) + [(\psi_t(\hat{\nu}) \cdot f_t) \hat{\star}_H U](\nu, g)\right) \quad (79)$$

$$(\text{by inductive hyp.}) = \sigma\left(\psi_1(\nu) \cdot [(\psi_{t-1}(\hat{\nu}) \cdot h_t[f_{<t}]) \star_H W](\nu - \hat{\nu}, g) + [(\psi_t(\hat{\nu}) \cdot f_t) \hat{\star}_H U](\nu, g)\right) \quad (80)$$

$$(\text{by trivial inp. lift}) = \sigma\left(\psi_1(\nu) \cdot [(\psi_{t-1}(\hat{\nu}) \cdot h_t[f_{<t}]) \star_H W](\nu - \hat{\nu}, g) + [(\psi_t(\hat{\nu}) \cdot f_t) \hat{\star}_H U](\nu - \hat{\nu}, g)\right) \quad (81)$$

$$(\text{by equiv. flow-conv}) = \sigma\left(\psi_1(\nu) \cdot \psi_{t-1}(\hat{\nu}) \cdot [h_t[f_{<t}] \star_H W](\nu - \hat{\nu}, g) + \psi_t(\hat{\nu}) \cdot [f_t \hat{\star}_H U](\nu - \hat{\nu}, g)\right) \quad (82)$$

$$(\text{by flow properties}) = \sigma\left(\psi_t(\hat{\nu}) \cdot \psi_1(\nu - \hat{\nu}) [h_t[f_{<t}] \star_H W](\nu - \hat{\nu}, g) + \psi_t(\hat{\nu}) \cdot [f_t \hat{\star}_H U](\nu - \hat{\nu}, g)\right) \quad (83)$$

$$(\text{by equiv. non-lin.}) = \psi_t(\hat{\nu}) \cdot \sigma\left(\psi_1(\nu - \hat{\nu}) [h_t[f_{<t}] \star_H W](\nu - \hat{\nu}, g) + [f_t \hat{\star}_H U](\nu - \hat{\nu}, g)\right) \quad (84)$$

$$(\text{by FERNN Eqn. 11}) = \psi_t(\hat{\nu}) \cdot h_{t+1}[f](\nu - \hat{\nu}, g) \quad (85)$$

$$= h_{t+1}[f](\nu - \hat{\nu}, \psi_t(\hat{\nu})^{-1} \cdot g), \quad (86)$$

Thus, assuming the inductive hypothesis for time t implies the desired relation at time $t + 1$; together with the base case this completes the induction and proves Theorem 4.1. $\square$

B Experiment Details

In this section we describe the datasets, models, training and evaluation procedures used in §5 of the main text. We additionally include samples from each dataset for visualization. The full code to reproduce these experiments is available at: <https://github.com/Neurips-FERN/FERN>

B.1 Flowing MNIST: Dataset Creation

As described in the main text, we construct sequences from the Flowing MNIST dataset by applying a flow generators ν randomly picked from an admissible set V_{train} , V_{val} , & V_{test} to samples from the corresponding train / validation / test split of the original MNIST dataset [LeCun et al., 1998]. The training sequences are always composed of $T = 20$ time-steps, and identically for the test sequences (except in the length-generalization experiments where the test sequence length is increased to 40 time-steps). We similarly generally set $V_{train} = V_{val} = V_{test}$, except in the velocity generalization experiments, where we specify the training and test flows explicitly. Note that the set V which defines the set of flows to which the FERNs are equivariant can be anything, and must not match the V_{train} of the dataset. Explicitly, denoting the value of the i th MNIST training image at pixel coordinate (x, y) as $m_{train}^{(i)}((x, y))$, a sample from the training set can be written as:

$$f_t^{(i)}((x, y)) = \psi_t(\nu) \cdot m_{train}^{(i)}((x, y)) \quad \forall (x, y) \in \mathbb{Z}^2, t \in \mathbb{Z}_+ \leq T \quad \text{where } \nu \sim V_{train} \quad (87)$$

We note that above, as in the main paper, we have written the signals and the kernels on the infinite domain $\mathbb{Z}^2$. In practice, however, both are only non-zero on a small portion of this domain (i.e. between $(0, 0)$ & $(28, 28)$ for MNIST digits). We do this to simplify the analysis, analogous to prior work [Cohen and Welling, 2016a].

Translating MNIST. For the planar 2D-translation group variant of Flowing MNIST, we consider the group

$$G = T(2) = (\mathbb{Z}^2, +), \quad \text{with binary operation} \quad (x, y) + (x', y') = (x + x', y + y'). \quad (88)$$

We use flow generators which are elements of the Lie algebra of this group, $\nu \in \mathfrak{t}(2)$, which we similarly denote as vectors in $\mathbb{Z}^2$, with the Lie bracket $[\gamma, \nu] = 0 \quad \forall \gamma, \nu \in \mathfrak{t}(2)$, meaning the translations are commutative. We note that in a matrix representation, we embed the elements of $T(2)$ into the affine group with homogeneous 3×3 matrices:

$$(x, y) \mapsto \begin{pmatrix} 1 & 0 & x \\ 0 & 1 & y \\ 0 & 0 & 1 \end{pmatrix}, \quad (89)$$

where the corresponding Lie algebra elements are

$$X(\nu) = \begin{pmatrix} 0 & 0 & \nu_1 \\ 0 & 0 & \nu_2 \\ 0 & 0 & 0 \end{pmatrix}, \quad \nu = (\nu_1, \nu_2) \in \mathbb{Z}^2. \quad (90)$$

The exponential map is then simply: $\exp(X(\nu)) = I + X(\nu)$. We see that the flow is then given as: $\psi_t(\nu) = \exp(tX(\nu)) = I + tX(\nu)$, and the action of the flow on a given pixel coordinate $g = (x, y)$ is given as:

$$\psi_t(\nu) \cdot g = t \begin{pmatrix} 1 & 0 & \nu_1 \\ 0 & 1 & \nu_2 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & x \\ 0 & 1 & y \\ 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & x + t\nu_1 \\ 0 & 1 & y + t\nu_2 \\ 0 & 0 & 1 \end{pmatrix}, \quad (91)$$

i.e. a shift of the pixel coordinates by velocity ν for t time-steps. In practice, order to be able to generate long sequences without excessively large images, we perform all translations with cyclic boundary conditions on our input and hidden states, i.e. $\psi_t(\nu) \cdot (x, y) = ((x + t\nu_1) \bmod W, (y + t\nu_2) \bmod H)$, for image size (H, W) . Since all of our convolutions are also performed with cyclic boundary conditions, this does not impact performance.

In our experiments, to define the sets of generators that we are interested in, we always use the integer lattice up to velocities $N \frac{\text{pixels}}{\text{step}}$. We denote these sets with the notation:

$$V_N^T := \{\nu \in \mathbb{Z}^2 \mid \|\nu\|_\infty \leq N\}. \quad (92)$$

For example V_2^T is the set of all 2D translation vectors with maximal velocity component ± 2 in either dimension, i.e. $V_2^T = \{(-2, -2), (-2, -1), (-2, 0), \dots, (2, 2)\}$.

Rotating MNIST. For the planar rotation variant of Flowing MNIST, we consider the group

$$G = SO(2) = (\mathbb{R}, +), \quad \text{with binary operation} \quad \theta + \theta' = (\theta + \theta') \bmod 2\pi. \quad (93)$$

Elements of the Lie algebra $\mathfrak{so}(2)$ are one-parameter generators of in-plane rotations, each of the form $\nu = \omega J$, where $J = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$ and $\omega \in \mathbb{Z}$ denotes the (integer-scaled) angular velocity. Because $\mathfrak{so}(2)$ is abelian, the Lie bracket $[\gamma, \nu] = 0 \quad \forall \gamma, \nu \in \mathfrak{so}(2)$.

In homogeneous coordinates we embed $SO(2)$ into the affine group with 3×3 matrices:

$$\theta \mapsto \begin{pmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{pmatrix}, \quad (94)$$

whose corresponding Lie-algebra elements are

$$X(\nu) = \begin{pmatrix} 0 & -\omega & 0 \\ \omega & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}, \quad \nu = \omega J, \quad \omega \in \mathbb{Z}. \quad (95)$$

Since $X(\nu)^2 = -\omega^2 I_{2 \times 2}$, the exponential map is the usual matrix exponential for planar rotations:

$$\exp(X(\nu)) = I + \sin(\omega) J + (1 - \cos(\omega)) J^2.$$

Hence the flow generated by ν is $\psi_t(\nu) = \exp(t X(\nu))$, and its action on a pixel coordinate $g = (x, y)$ (in homogeneous form) is

$$\psi_t(\nu) \cdot g = \begin{pmatrix} \cos(t\omega) & -\sin(t\omega) & 0 \\ \sin(t\omega) & \cos(t\omega) & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} = \begin{pmatrix} x \cos(t\omega) - y \sin(t\omega) \\ x \sin(t\omega) + y \cos(t\omega) \\ 1 \end{pmatrix}, \quad (96)$$

i.e. a rotation about the image centre by angle $t\omega$.

Following our experimental protocol, we discretize the angular velocity at $\Delta\theta = 10^\circ$ intervals and collect the set of generators

$$V_N^R := \{ \nu = k \Delta\theta J \mid k \in \mathbb{Z}, |k| \leq N \}. \quad (97)$$

For instance, $V_4^R = \{-40^\circ, -30^\circ, -20^\circ, -10^\circ, 0^\circ, 10^\circ, 20^\circ, 30^\circ, 40^\circ\}$. In practice, to implement the spatial rotation we use the Pytorch function `F.grid_sample` with zero-padding and bilinear interpolation. We additionally zero-pad all images with 6-pixels on each side (resulting in images of size (40×40)) to allow for the rotation to fit within the full image frame.

B.2 Flowing MNIST: Models

For the Flowing MNIST datasets we compare three types of models: standard group-equivariant RNNs (G-RNNs) as defined in Equation 4, FERNNs with equivariance to a subset of the training flows (i.e. $V_{\text{model}} = V_1^T$ and $V_{\text{train}} = V_2^T$), and FERNNs with full equivariance to the training flows ($V_{\text{model}} = V_{\text{train}}$). For all models on each dataset we use the same model architecture, and since there are no extra parameters introduced by the FERNN model, all models have the same number of trainable parameters.

Translating MNIST G-RNN. For the translation group, the corresponding group-convolution is the standard 2D convolution. We therefore build a G-RNN exactly as written in Equation 4 with regular convolutional layers in place of the group-convolution. We use kernel sizes of 3×3 for both $\mathcal{U}$ and $\mathcal{W}$ with no bias terms, strides of 1 and circular padding of 1, resulting in a hidden state with spatial dimensions equal to the input spatial dimensions: 28×28 . We use 128 output channels for our convolutional ‘encoder’ $\mathcal{U}$, and similarly 128 input and output channels for our recurrent kernel $\mathcal{W}$. This results in a hidden state $h \in \mathbb{R}^{128 \times 28 \times 28}$. We use a ReLU activation function: $\sigma(h) = \max(0, h)$. We initialize all hidden states to zero: $h_0 = \mathbf{0}$, satisfying our equivariance proof assumptions. At each timestep, we decode the updated hidden state to predict the next input through a 4-layer CNN decoder: $g_\theta(h_{t+1}) = \hat{f}_{t+1}$. The CNN decoder is composed of three primary convolutional layers each with 128 input and output channels, kernel size 3×3 , stride 1, padding 1, followed by ReLU activations. A final identical convolutional layer, but with 1 output channel, is used to predict $\hat{f}_{t+1}$.

Translating MNIST FERNs. For the FERNs, we use the exact same RNN and decoder architecture, with the only difference being that we extend the hidden state with an extra V dimension for each of the flows in V_{model} : $h(\nu, g)$. Explicitly then, through the trivial lift, the input is copied identically to each of these ν channels, and the flow convolution on the hidden state also operates identically on each ν channel. As stated in the main text, we define the flow convolution to not mix the V channels at all, explicitly: $\mathcal{W}_k^i(\nu, g) = \delta_{\nu=e} \mathcal{W}_k^i(g)$, thus giving us constant parameter count. Our lifted hidden state is thus $h \in \mathbb{R}^{|V_{model}| \times 128 \times 28 \times 28}$. The flow action $\psi_1(\nu) \cdot$ in the recurrence is implemented practically as a Roll of the hidden state tensor by (ν_1, ν_2) steps along the (x, y) spatial dimensions. To achieve invariance of the reconstruction to the input flows, and thereby achieve the generalization we report, we max-pool over the V dimensions before decoding. Explicitly, the output of the model for each time step is computed as: $g_\theta(\max_\nu h_{t+1}(\nu, g)) = \hat{f}_{t+1}$.

Rotating MNIST G-RNN. For the rotating MNIST models, we use a nearly identical setup to the translation experiments, with the only difference being that we use $SE(2)$ group convolutions in place of the standard convolutional layers to achieve the necessary rotation equivariance. In practice, we use the *escnn* library to implement the $SE(2)$ convolutions [Cesa et al., 2022]. We discretize the rotation group into $\Delta\theta = 10^\circ$ rotations, yielding a cyclic group with 36 elements: C_{36} . We lift the input to this space and assert a regular representation of the group action on the output. Due to the increased dimensionality of the hidden state from the lift to the discrete rotation group, we decrease the number of hidden state channels to 32 due to hardware limitations. This then yields a hidden state: $h \in \mathbb{R}^{36 \times 32 \times 40 \times 40}$.

Rotating MNIST FERNs. For the FERN models, we follow the same procedure as for the translating MNIST FERNs, except with the action of the group $\psi_1(\nu) \cdot$ in the recurrence now taking the form of the regular representation of rotation in the $SE(2)$ equivariant CNN output space. Explicitly, this means that in addition to rotating the inputs, we also permute along the lifted rotation channel for each angular velocity ν . Again this yields a hidden state of size: $h \in \mathbb{R}^{|V_{model}| \times 36 \times 32 \times 40 \times 40}$.

B.3 Flowing MNIST: Next-Step Prediction Training & Evaluation

For the in-distribution next step prediction experiments, (Table 1 and Figure 2) of the main text, we use $V_{train} = V_{val} = V_{test} = V_2^T$ for the translating MNIST experiments, and $V_{train} = V_{val} = V_{test} = V_4^R$ for the rotating MNIST experiments. We set the training sequence length to 20 steps, providing the models with 10 time-steps as input, and computing the next-step prediction reconstruction loss (MSE) of the model output on the remaining 10 time-steps. Explicitly:

$$\mathcal{L} = \frac{1}{10} \sum_{t=11}^{20} \|f_t - g_\theta(h_t)\|_2^2 \quad (98)$$

During training, we use ‘teacher forcing’ [Williams and Zipser, 1989] with 50% probability, meaning that at each time-step of the 10 forward prediction steps, the model has a 50% probability of receiving the true input as f_t , and a 50% probability of receiving it’s own autoregressive prediction $\hat{f}_t$ as input. This technique is designed to improve model convergence of autoregressive prediction models by avoiding loss plateaus induced by poor forward prediction. Note that this procedure is not used during validation or testing, ensuring pure autoregressive next-step performance is evaluated.

All models are trained for 50 epochs, with a learning rate of 1×10^{-4} using the Adam optimizer Kingma and Ba [2017]. For translation flows, we use a batch size of 128, and clip gradient magnitudes at 1 in all models for additional stability. For rotation flows we use a batch size of 32 due to memory constraints, and find gradient clipping not necessary. For evaluation, we save the model with the best performance on the validation set (over epochs), and report its corresponding performance on the held-out test set. For each model we train with 3 random initializations (3 seeds) and report the mean and standard deviation of the test set performance from the best saved models.

Length Generalization. For the out-of-distribution length generalization experiments (Figure 3) we again use $V_{train} = V_{val} = V_{test} = V_2^T$ for the translating MNIST experiments, and $V_{train} = V_{val} = V_{test} = V_4^R$ for the rotating MNIST experiments. Where the length of training and validation sequences is again set to $T = 20$ as before. At test time, we increase the length of the sequences to

$T = 40$, but continue to only feed the models the first 10 time-steps as input. In Figure 3 we show the loss of the model for each of these remaining 30 time steps ahead.

Velocity Generalization. For the out-of-distribution velocity generalization experiments (Figure 4), we leave the train and test sequence length at 20, but we instead set $V_{train} = V_{val} \subset V_{test}$. Explicitly, in Figure 4, for rotating MNIST, we set $V_{train} = V_{val} = V_1^R$ and $V_{test} = V_5^R$. For translating MNIST, we set $V_{train} = V_{val} = V_1^T$ and $V_{test} = V_2^T$. This tests the ability of models to generalize to new flows not seen during training, and we see that the FERNs perform significantly better on this test set when they are made equivariant to these velocities.

B.4 Moving KTH: Datasets

To test the benefits of flow equivariance on a sequence classification task with real image sequences, we opted to use the KTH action recognition dataset [Schuldt et al., 2004], obtained from <http://www.csc.kth.se/cvap/actions/>. The dataset is composed of 2391 videos of 25 people performing 6 different actions: running, jogging, walking, boxing, hand clapping, and hand waving. The original videos are provided at a resolution of 160×120 with 25 frames per second, and an average clip length of 4 seconds. The training, validation, and test sets are constructed from this dataset by taking the videos from the first 16 people as training, the next 4 people as validation, and the last 5 people as test. We split the videos into clips of 32-frames each, downsample to a spatial resolution of 32×32 , and subsample them by half in time, yielding a final set of clips which are 16 steps long.

Since the videos from this dataset are taken entirely from a stationary camera viewpoint, there are no global flows of the input space which our model might benefit from. We call this original dataset KTH with V_0^T (no motion). To test the benefits of flow equivariance in an action recognition setting with a moving viewpoint, we construct two additional variants of the KTH dataset, augmented by translation flows from the sets V_1^T and V_2^T . These sets are identical to those described in the Flowing MNIST examples: translation with circular boundary conditions. Again, since we use convolution with circular boundary conditions in all our models, this does not impact model performance.

B.5 Moving KTH: Models

We compare five models on the KTH dataset: a 3D-CNN, a G-RNN, two FERN variants, and an ablation of the FERN (denoted G-RNN+). We describe these in detail below:

Moving KTH 3D-CNN. The spatio-temporal 3D-CNN baseline is built as a sequence of five 3D convolution layers, interleaved with 3D batchnorm and ReLU activations. Each layer has kernels of shape $(3 \times 3 \times 3)$, no bias, and padding 1. The first layer has 16 output channels, a temporal stride of 2, and a spatial stride of 1. Layer 2 has 32 output channels, temporal stride of 1, and spatial stride of 2. Layer 3 has 32 output channels, temporal stride of 1, and spatial stride of 1. Layer 4 has 64 output channels, temporal stride of 1, and spatial stride of 2. Layer 5 has 64 output channels, temporal stride of 1, and spatial stride of 1. This final layer is followed by a global average pooling over the remaining $(8 \times 8 \times 8)$ space-time feature map dimensions, yielding a single vector of dimensionality 64 which is passed through a linear layer to predict the logits for the 6 classes.

Moving KTH G-RNN. For the baseline G-RNN, each grayscale input frame $f_t \in \mathbb{R}^{1 \times 32 \times 32}$ is passed through a three-layer convolutional encoder that preserves spatial resolution:

$$\text{Conv}_{5 \times 5}^{1 \rightarrow 32} \xrightarrow{\text{BN+ReLU}} \text{Conv}_{3 \times 3}^{32 \rightarrow 64} \xrightarrow{\text{BN+ReLU}} \text{Conv}_{3 \times 3}^{64 \rightarrow 128} \xrightarrow{\text{BN+ReLU}},$$

all with stride 1 and circular padding chosen so that the hidden state has the same spatial dimensions as the input (32×32). The output defines the encoder feature map $\mathcal{U} * f_t$. The hidden state $h_t \in \mathbb{R}^{128 \times 32 \times 32}$ is updated with a single recurrent layer using a circularly padded 3×3 convolution where $\mathcal{W} \in \mathbb{R}^{128 \times 128 \times 3 \times 3}$ contains no bias terms. The recurrent convolution is also followed by a batch-norm and ReLU non-linearity for added expressivity. The non-linearity σ is tanh. Initial states are zero, $h_0 = \mathbf{0}$, satisfying the assumptions of our equivariance proof. At the final timestep of the sequence ($t = 16$) we take the hidden state of RNN, perform global average pooling to a 1×1 spatial dimensions, and feed the resulting 128-dimensional vector through a fully-connected layer:

$$g_\theta(h_T) = \text{FC}_{128 \rightarrow 6}(\text{SpatialAvgPool}(h_T)) \in \mathbb{R}^6,$$

producing logits for the six KTH action classes.

Moving KTH FERNNs. For the FERNNs, we use the exact same architecture, but use the trivial lift to lift the corresponding sets of flows V_1^T for FERNN- V_1^T and V_2^T for FERNN- V_2^T (denoted FERNN-1 and FERNN-2 in Figure 6). Concretely, for every translation velocity $\nu \in V_{\text{model}}$ we allocate an additional velocity channel, so that the hidden state becomes $h_t(\nu, g) \in \mathbb{R}^{|V_{\text{model}}| \times 128 \times 32 \times 32}$, where $g = (x, y)$ indexes spatial position. Input frames are trivially lifted by copying the same encoder features into each ν -channel. Following Equation 11, the flow action in the recurrence is implemented again as a Roll of the spatial dimensions of the hidden tensor by (ν_x, ν_y) pixels, and the flow convolution uses weight sharing across velocities, $\mathcal{W}_k^i(\nu, g) = \delta_{\nu=e} \mathcal{W}_k^i(g)$, so the total number of trainable parameters is identical to the G-RNN. As in the Flowing-MNIST experiments, we consider two settings: (i) *partial* equivariance with $V_{\text{model}} = V_1^T \subset V_{\text{train}} = V_2^T$ and (ii) *full* equivariance where $V_{\text{model}} = V_{\text{train}}$. To achieve flow-invariant action classification we take the maximum over the velocity dimension after the final FERNN layer, $\max_{\nu} h_t(\nu, g)$, followed by the same global-average-pooling and linear classifier used for the G-RNN. This design ensures that any translation-induced shifts present at test time are pooled over, yielding the generalization results reported in Figure 6.

Moving KTH G-RNN+. To ensure that the observed performance improvement of the FERNN was not simply due to the increased number of hidden state activations and the associated max-pooling, but instead could be attributed the precise flow equivariant form of the recurrence introduced in Equation 11, we built a third baseline which is as close as possible to the best performing FERNN (FERNN- V_2^T), while removing precise flow equivariance. Specifically, while keeping all other architectural components of the FERNN- V_2^T identical, we replaced the single-step action of the flow in the recurrence ($\psi_1(\nu) \cdot$) with convolution by a separate learned 5×5 convolutional kernel for each ν channel (randomly initialized). Since the action of $\psi_1(\nu)$ is a simple translation (a local linear operation), this can indeed be represented by such a kernel. However, as we see in practice (Figure 6), the model fails to learn such kernels and instead overfits to the training data distribution.

B.6 Moving KTH: Action Recognition Training & Evaluation

Models are trained to minimize the cross entropy loss between the predicted class and the ground truth label using the Adam optimizer. Due to the small dataset size, all models are trained for 500 epochs, with a batch size of 32. We search over learning rates in the set $\{3 \times 10^{-3}, 1 \times 10^{-3}, 3 \times 10^{-4}, 1 \times 10^{-4}\}$, for each model, running three random initialization seeds for each. For each random seed of each hyper-parameter setting, we store the model with the best validation loss. We then pick the best performing model based on the mean value of the best validation loss across all three seeds. We then report in Table 2 the mean test loss of the models (3 seeds) saved at the best validation loss epoch for the best identified learning rate. We generally find the lower learning rates (3×10^{-4}) work better for the RNN models, while the higher learning rates (1×10^{-3}) work better for the CNNs.

B.7 Moving KTH Samples

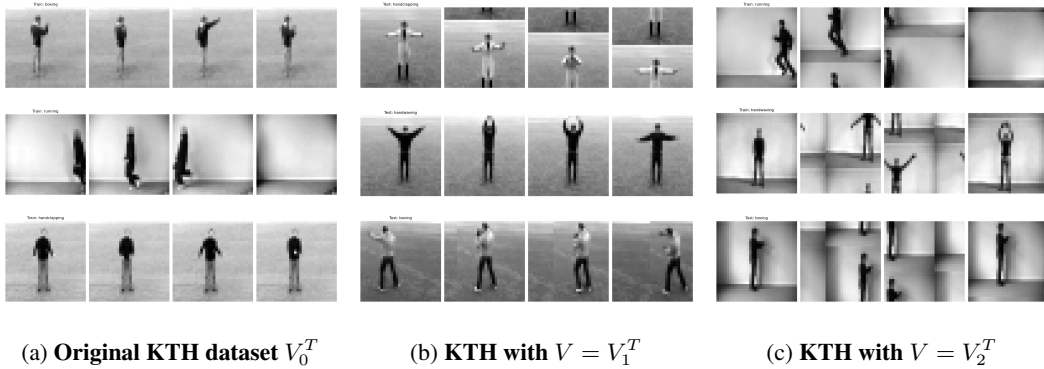


Figure 7: Samples of the original KTH dataset and its two motion-augmented variants.

B.8 Compute

All experiments in this paper were performed on a private cluster containing a mixture of NVIDIA A100 and H100 GPUs, each having 40GB and 80GB of VRAM respectively. No parallelization of individual models across GPUs was required, i.e. most models and training paradigms were able to fit on a single A100 GPU, with the larger models on a single H100 GPU. The cluster nodes allocated up to 24 CPU cores and 375GB of RAM per job, although only a small fraction of this was required for training and evaluation. The majority of models were able to train fully in less than 24 hours. For example, the FERNN- V_2^T models on KTH trained in 7 hours, and the FERNN- V_2^T models on MNIST trained in 15 hours, with all other models training faster. The significant exception to this were the FERNN- V_4^R models on Rotating MNIST which took roughly 67 hours to complete 50 epochs (although they converged much more quickly than this, see Figure 2, we ran them to the same number of epochs as the G-RNN for consistency). The reason for this increased computational time was an inefficient implementation of the rotation operation and our custom recurrence, which could both be accelerated in future work. Specifically, we used a naive vanilla Pytorch implementation of our custom FERNN recurrence (Equation 11) using ‘for loops’, which dramatically slowed down training for all models. In future work, implementation of the model with a scan operation in JAX, or a custom CUDA kernel would dramatically improve runtime performance. Overall, we estimate the computational requirements necessary to develop the models and run all experiments for this paper totaled approximately 30 days of H100 compute time.

C Variations of FERNNs

C.1 FERNN with Non-Trivial Lift

As noted in §4, it is possible to build a FERNN with a non-trivial lift, such that the lifting convolution itself incorporates the flow transformation for each ν dimension.

Explicitly, we can define such a lift as:

$$[f_t \hat{\star}_{V \times G} \mathcal{U}^i](\nu, g) = \sum_{x \in X} \sum_{k=1}^K f_k(x) \mathcal{U}_k^i(g^{-1} \cdot \psi_t(\nu)^{-1} \cdot x) \quad (99)$$

Another way to think of this, is that there is a time-parameterized input kernel defined on the full space $V \times G$, i.e. $\hat{\mathcal{U}}(\nu, g, t) = \mathcal{U}(\psi_t(\nu)^{-1} \cdot g)$; however, we find this viewpoint less elegant given that the kernel then depends on time.

We see that when the flow is incorporated into the lifting convolution, the output of the convolution is no longer constant along the ν index (as it was in the trivial lift). Instead, it is now flowing according to the ν 'th vector field. Therefore, when processing an input undergoing a given flow $\psi(\hat{\nu})$, this input flow will combine with the flows of the lifting convolution to yield a shift along the V dimensions, similar to what we previously observed in the hidden state, i.e.:

$$\begin{aligned} [(\psi_t(\hat{\nu}) \cdot f_t) \hat{\star}_{V \times G} \mathcal{U}^i](\nu, g) &= \sum_{x \in X} \sum_{k=1}^K f_k(\psi_t(\hat{\nu})^{-1} \cdot x) \mathcal{U}_k^i(g^{-1} \cdot \psi_t(\nu)^{-1} \cdot x) \\ &= \sum_{\hat{x} \in X} \sum_{k=1}^K f_k(\hat{x}) \mathcal{U}_k^i(g^{-1} \cdot \psi_t(\nu - \hat{\nu})^{-1} \cdot \hat{x}) \quad (\text{where } \hat{x} = \psi_t(\hat{\nu})^{-1} \cdot x) \end{aligned} \quad (100)$$

$$= [f_t \hat{\star}_{V \times G} \mathcal{U}^i](\nu - \hat{\nu}, g) \quad (102)$$

Notably then, keeping the flow convolution from Equation 9 unchanged, we see that we must remove the additional $\psi_1(\nu)$ shift from the original FERNN in order to maintain flow equivariance. Specifically, the new non-trivial-lift recurrence relation is then given simply as:

$$h_{t+1}(\nu, g) = \sigma([h_t \star_{V \times G} \mathcal{W}](\nu, g) + [f_t \hat{\star}_{V \times G} \mathcal{U}](\nu, g)). \quad (103)$$

In this setting, the action of the flow on the output space changes to just a permutation of the V dimension, with no corresponding flow on g :

$$(\psi(\hat{\nu}) \cdot h[f])_t(\nu, g) = h_t[f](\nu - \hat{\nu}, g) \quad (104)$$

In a sense, this model can be seen as ‘undoing’ the action of each flow on the input when lifting. We find this to be somewhat analogous to the traditional group-equivariant CNN design choice where the transformation can *either* be applied to the filter or the input. In the FERNN setting, the ‘filter’ is now defined by the full recurrence relation, so we can either apply the flow transformation to the input sequence, or to the hidden state sequence.

Overall, we find this to be a slightly less elegant construction since the indexing of the kernel in the convolution is then dependent on the time index explicitly. In the trivial-lift setting introduced in the main text, this time-dependence is rather implicitly imposed by the recurrence of the hidden state itself, therefore allowing us to only require the instantaneous one-step flows during each recurrent update. Regardless, we are interested in future work which may explore this non-trivial lift setting more fully, and other interpretations of the FERNN model as described.

D Related Work

In this section we provide an overview of work which is related to flow equivariance and equivariance with respect to time-parameterized symmetries generally.

D.1 Flow Equivariance without a Hidden State

As mentioned in the main text, it is possible to achieve flow equivariance without an explicitly flow-equivariant sequence model. The two primary methods for accomplishing this are through frame-wise application of an equivariant model (as described in §3) and through group-convolution over the entire space-time block (as described in §6). Both of these methods are verifiably flow-equivariant, however they are fundamentally a different class of model than what we have described in this work. They are not recurrent sequence models, and therefore intrinsically have a finite temporal context or ‘receptive field’ which can be used when computing any output. By contrast, recurrent networks can theoretically have an infinite temporal context, if need be, through the maintenance of a hidden-state. Given this is a fundamental distinction between recurrent and non-recurrent networks which is the subject of research beyond the domains of equivariance, we find it to be beyond the scope of this work to compare with these models explicitly. Instead, we propose flow equivariant RNNs as an extension of existing equivariant network theory to this class of models which maintain a hidden state and can operate in the online recurrent setting.

The list of prior work which can be included in this category of ‘flow equivariance without a hidden state’ is quite broad, since it encompasses most of the equivariant deep learning literature to date, however we list a few notable examples here. The Lorentz equivariant work of Bogatskiy et al. [2020], Gong et al. [2022] is the most relevant, while other applied work has developed 3-D convolutional networks which are equivariant with respect to Galilean shifts (our translation flows) in the context of event-based cameras [Zhu et al., 2019], or rotations over time in the context of medical imaging [Zhu et al., 2024]. Early work developed Minkowski CNNs [Choy et al., 2019], which are equivariant with respect to 4D translations, thus making them equivariant to axis-aligned motions. Further, Clifford Steerable CNNs [Zhdanov et al., 2024] have also been developed to achieve Poincaré-equivariance on Minkowski spacetime. Related work on equivariance for PDE solving / forecasting has built dynamics models which are equivariant with respect to galilean transformations in the sense that the model is equivariant if the input vector field has a global additive constant [Wang et al., 2021]. While this is valid for neural networks applied to vector field data as input, it is clearly not the same as our method in more general settings. The method of Wang et al. [2021] can be interpreted as viewing a dynamical system which has an unknown global current introduced, while ours is better interpreted as viewing the dynamical system from a moving reference frame – the two concepts are compatible and may even be combined.

D.2 ‘Statically Equivariant’ Sequence Models

The second broad category of related work includes sequence models which are equivariant with respect to instantaneous static group transformations, but are not equivariant with respect to time-parameterized group transformations, as our FERNN is. Examples in this category include [Azari and Erdoğan, 2022, Nguyen et al., 2023, Basu et al., 2023], which introduce equivariance to transformations such as static rotations in sequence models including RNNs. For example, these models train on one frame of reference and then test on a ‘rotated’ frame of reference. Our work can be seen to generalize these models to instead allow them to be tested on ‘rotating’ frames of reference. This class of prior work can most readily be compared to the group equivariant RNN (G-RNN) we describe in §2. Other researchers have developed sequence to sequence models which are equivariant with respect to fixed permutations and demonstrated that this is beneficial in the context of language modeling [Gordon et al., 2020]. Further, recent work has developed an equivariant sequence autoencoder which uses group convolutions in an LSTM to achieve static equivariance for the purpose of PDE modeling [Fromme et al., 2025].

D.3 Neuroscience and Biologically Inspired Neural Networks

The study of symmetry has also grown increasingly relevant in the computational neuroscience literature. Zhang et al. [2022] have studied equivariant representations in biological systems in

terms of continuous attractor networks. Interestingly, these models also use convolutional recurrent dynamics; however, again, this work only considers static translations and not motion over time. Related work in the domain of neuroscience has studied how waves can be used to represent motion explicitly [Shaw and Kilpatrick, 2023]. Interestingly, these waves can be seen to be instantiations of the regular representation of the group action that is used in the hidden state of our FERNN model (see [Keller et al., 2024b] for a description of the high level intuition for how equivariance, regular representations, and waves are related). Waves have also been shown to model visual illusions [Barch et al., 2010, Chemla et al., 2018], and some models have even demonstrated bow-wave like effects in neural fields [Glaser and Barch, 1999, Barch and Glaser, 2002], arguing that these effects are potentially beneficial for motion detection. Scientists have also measured doppler shifts in neural responses as a function of moving stimuli [Zhang et al., 2016]. Finally, traveling waves in visual cortex are also known to be induced by movement, such as saccades [Zanos et al., 2015], an effect which we believe holds some relation to the representation of the flow action on our FERNN’s hidden state.

One class of models which is highly related to the FERNN in both theory and implementation comes from a line of biologically inspired work aiming to learn symmetries from data. One of the first models in this category, the Topographic VAE [Keller and Welling, 2022] can be seen as similar to our FERNN but with an explicitly imposed translation flow of a single velocity in the latent space. This makes these models equivariant to input transformations which are isomorphic to the translation group on the integers modulo the capsule length. Interestingly, the authors find that by simply imposing this translation flow in the latent space, the model learns to encode dataset symmetries into these flows in order to better model the dataset. This seems to imply that simply imposing flow symmetries in sequence models without a priori knowledge of the structure of the flow symmetries in the input may still be beneficial. Similar results were shown with the Neural Wave Machine [Keller and Welling, 2023], where flows in the latent space were implemented implicitly through a bias towards traveling wave dynamics. Finally, perhaps most interestingly, related work on traveling waves in simple recurrent neural networks (the wave-RNN) [Keller et al., 2024a] and SSMs [Keller, 2025] has demonstrated that waves implemented through similar ‘roll’ operations have significant benefits for long-term memory in recurrent neural network architectures. Incredibly, these models are identical to translation flow-equivariant RNNs in implementation, but without any mention of equivariance, and applied to an entirely different set of tasks. It is therefore of great interest to study if there is something unique to translation flows which benefit memory performance, or if similar performance benefits may be gained from any latent flow symmetry.

D.4 Reference Frames in Neural Networks

As mentioned in §4, one way to interpret the flow-equivariant RNN is that its hidden state lives in a number of moving reference frames simultaneously (one for each $\nu \in V$). Thus, for moving inputs, the corresponding co-moving hidden state reference frame will see the input as stationary, and process it as normal. This idea of reference frames in neural networks is not new, and significant interesting related work should be noted.

Specifically, Spatial Transformer Networks [Jaderberg et al., 2016], and Recurrent Spatial Transformer Networks [Sønderby et al., 2015] can be seen to predict a frame of reference for a given input and then switch to that reference frame to gain invariance properties. However, these models do not discuss moving reference frames. Other models have been built in this vein with respect to other symmetries, such as polar coordinate networks which are inherently equivariant with respect to scale and rotation [Esteves et al., 2018]. Related in theory is the idea of Capsule Networks [Sabour et al., 2017, Hinton et al., 2018]. These models do have an explicit notion of reference frames, similar to ours, and use this to gain a structured equivariant representation, but again this is defined only in the spatial context.

D.5 Broadly Related Work

More broadly, prior work has looked at the integration of recurrence and motion modeling specifically for vision [Wu et al., 2021, Gehrig and Scaramuzza, 2023]. These models do not have any mention of motion equivariance, and therefore are highly unlikely to provide the strong generalization benefits such as those that we present in this paper. Other work has studied ego motion for action recognition [López-Cifuentes et al., 2020]; and relevant work has looked at equivariance in the context of object tracking [Gupta et al., 2020, Sosnovik et al., 2020a].

D.6 Equivariant Dynamical Systems

In the dynamical systems literature, equivariance is typically defined for autonomous (or homogeneous) dynamical systems which have no ‘input’ or ‘driving force’. Abstractly, for a system $\frac{dx}{dt} = f(x)$, we say the dynamical system is equivariant if $f(g \cdot x) = g \cdot f(x)$ for all $g \in G$. Then for any $x(t)$ that solves the differential equation, we also know that $g \cdot x(t)$ solves the differential equation for the full group orbit $g \in G$ [Moehlis and Knobloch, 2007]. Similar to the equivariant neural network setting, we see that the ‘output’ of an equivariant dynamical system (interpreted as the particular solution) transforms in a predictable well-behaved manner for a given transformation of the input. As the simplest example, $\frac{dx}{dt} = x + x^3$, has a sign flip symmetry in x , meaning that if take the sign flipped version of the trajectory $x(t)$, the solution to the equation also inherits a sign flip. It is straightforward to see from the above definition that if the function $f(x)$ defining the time derivative is equivariant with respect to G , then the system is considered equivariant, since the definitions are equivalent.

In this work, we are interested in recurrent neural networks, which generally operate in the “forced” or non-autonomous setting. The study of symmetries in non-autonomous dynamical systems has been previously explored in the control theory literature, and has proven highly valuable for the design of robust and high performing equivariant filters [Mahony et al., 2022]. In this setting, the dynamical system is defined as $\frac{dx}{dt} = f(x, u)$ for some driving force u , and the equivariance property is then defined as $g \cdot f(x, u) = f(g \cdot x, g \cdot u)$. We see that our FERNNs indeed are equivariant non-autonomous dynamical systems by this definition.

The difference of our current work with this prior control-observer work, is that Mahony et al. [2022] treats time-parameterised symmetries as known biases inside a hand-written dynamical model and uses lifts/adjoint operators to keep the estimation error autonomous. The FERNN instead learns the dynamics, lifts the hidden state to a group-indexed field, and enforces equivariance by a simple group-convolution weight-sharing rule. This removes the need for handwritten dynamical models and allows learning flexible non-linear dynamics – capabilities not covered by existing observer theory.

E Extended Results: Flowing MNIST

E.1 In-Distribution Next-Step Prediction (Table 1 & Figure 2)

In Figure 8, we show the sequence predictions of the models presented in Figure 2 & Table 1, trained on Translating MNIST V_2^T and Rotating MNIST V_4^R , and evaluated on the same flows. We see that when tested in-distribution, all models appear to perform well from visual inspection. Full sequences of 20 steps (input and forward prediction steps) are presented, wrapped over 2 lines.

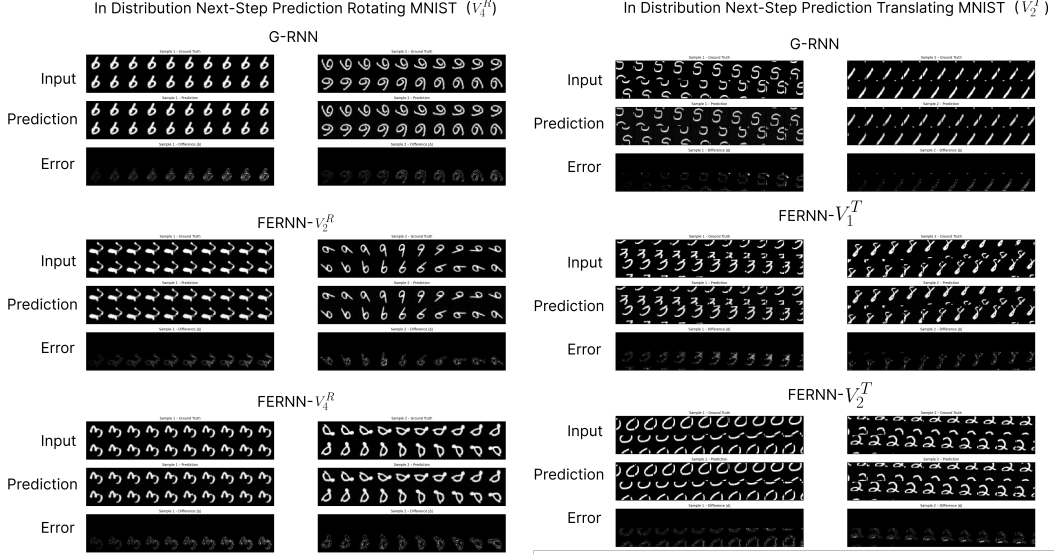


Figure 8: In-distribution sequence predictions for the models from Table 1 & Figure 2, trained on Rotating MNIST V_4^R (left) and Translating MNIST V_2^T (right), evaluated on the same flows.

E.2 Length Generalization Visualizations

In Figure 9, we show the length generalization plot, analogous plot to Figure 3 (right), but for Rotating MNIST. We see that the length generalization performance gap is not as significant on Rotating MNIST compared with Translating MNIST. We suspect that this is due to the accumulation of errors induced by repeated interpolation when performing rotation by small angles on a discrete grid. Despite this, we see that the FERNN- V_4^R still achieves strong generalization up to 20-time steps forward, significantly outperforming the non-equivariant model.

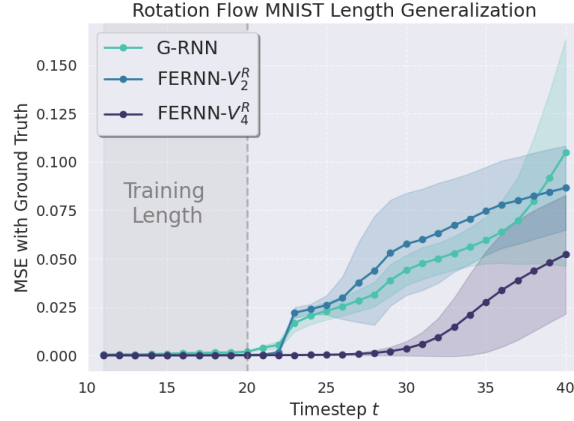


Figure 9: MSE vs. Forward prediction horizon for models on Rotating MNIST. Analogous plot to Figure 3 (right) but for Rotating MNIST.

In Figure 10 we show the sequence predictions of the same models presented in Table 1 & Figure 2, trained on Translating MNIST V_2^T and Rotating MNIST V_4^R , but tested in the length generalization setting. We plot the 30-forward prediction steps here, subsampled by half in time (giving 15 elements). We plot the ground truth sequences on top, and the forward predictions below, with the error (in blue-red color scheme) on bottom. We see the FERNNs significantly outperform the G-RNNs in length generalization on Translating MNIST, and also appear to perform noticeably better on Rotating MNIST, mirroring the quantitative results in Figures 9 & 3.

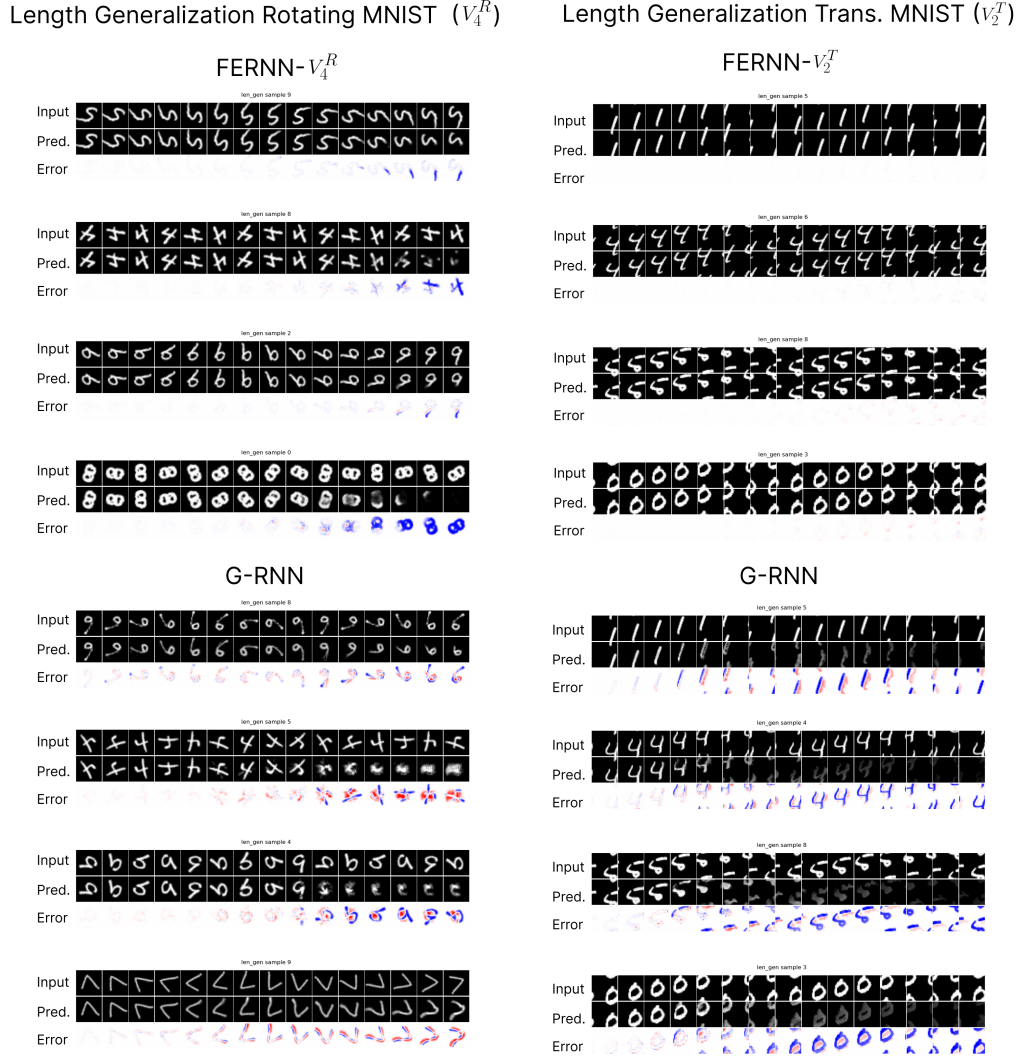


Figure 10: Samples from models trained on Rotating MNIST V_4^R (left) and Translating MNIST V_2^T (right) with training sequence lengths of 20, tested on sequence lengths of 40. We plot the 30-forward prediction steps here, subsampled by half in time (giving 15 elements).

E.3 Velocity Generalization Visualizations

In Figure 11 we show the sequence predictions of the models presented in Figure 4, trained on Translating MNIST V_1^T and Rotating MNIST V_1^R , but evaluated on V_2^T and V_4^R respectively. These figures show that FERNNs which are equivariant to flows beyond their training distribution are able to automatically generalize to these transformations at test time, achieving near perfect next-step prediction performance where non-flow equivariant RNNs fail.



Figure 11: Samples from models trained on Rotating MNIST V_1^R (left) and Translating MNIST V_1^T (right), and tested on sequences with significantly higher velocity flows (V_4^R and V_2^T respectively). We see that the FERNN (bottom rows) has no problem generalizing to these new velocities despite never having seen them in training, while the G-RNN (top) fails. The specific flows plotted in this example are $-40 \frac{deg}{step}$ for Rotating MNIST (left), and $\nu = (+2, +1)$ for Translating MNIST.